Golden Step

지금, 더 안전한 내일로

실종자 우선확인 수색 지원 서비스

팀명: 홉레이더

팀장: 이재훈

팀원: 김동규, 박철우

고용노동부 더조은컴퓨터아카데미

목차

01	프로젝트 개요	03p
02	프로젝트 팀 구성 및 역할	10p
03	프로젝트 수행 절차 및 방법	12p
04	프로젝트 수행 경과	16p
05	피드백 반영	34p
06	트러블 슈팅	40p
07	기술 연구	45p
08	형상관리 및 배포	55p
09	회의록	62p
10	자체 평가 의견	65p

01

프로젝트 개요

1.1. 프로젝트 주제 및 선정 배경, 기획 의도

1.2. 프로젝트 내용

1.3. 활용 장비 및 재료

1.4. 프로젝트 구조

1.5. 활용방안 및 기대 효과

1.1. 프로젝트 주제 및 선정 배경, 기획 의도

Golden Step

주제

보행 네트워크 기반 실종자 우선확인 수색 지원 서비스

선정 배경

실종·연락 두절 초기에는 확인 우선순위 판단이 어렵고, 단순 원형 반경 지도는 실제 이동환경을 반영하는 데 한계가 있음

기획 의도

마지막 확인 위치·시각과 실제 보행 도로망을 바탕으로 도달 가능 영역을 분석하고, 접근성과 체류 가능성에 따른 우선 확인 장소를 순위로 안내하여 경찰 수사를 대체하지 않는 생활안전 보조 도구를 제공

1.2. 프로젝트 내용

사용자 기능

- NAVER Maps API를 활용한 장소 검색 및 마지막 확인 위치 선택
- 마지막 확인 시각과 대상자 정보 입력을 통한 수색 분석 요청
- 지도 기반 예상 수색 범위 및 우선 확인 장소 시각화
- 현재 및 30분·1시간·3시간·6시간 이후의 예측 결과 조회
- 우선 확인 장소 목록 조회 및 확인 완료 상태 관리
- 수색보드를 통한 기존 수색 정보 조회 및 수색 이어가기
- 공유 링크를 통한 수색 결과와 장소 확인 상태 전달

수색 분석 기능

- 보행 네트워크 기반 이동 분석을 통한 예상 수색 영역 산출
- 대상자 정보와 경과 시간을 반영한 시간대별 수색 결과 생성
- 수색 시 우선적으로 확인할 장소 추천 및 순위 제공

1.3. 활용 장비 및 재료 개발환경

FRONT

BACK

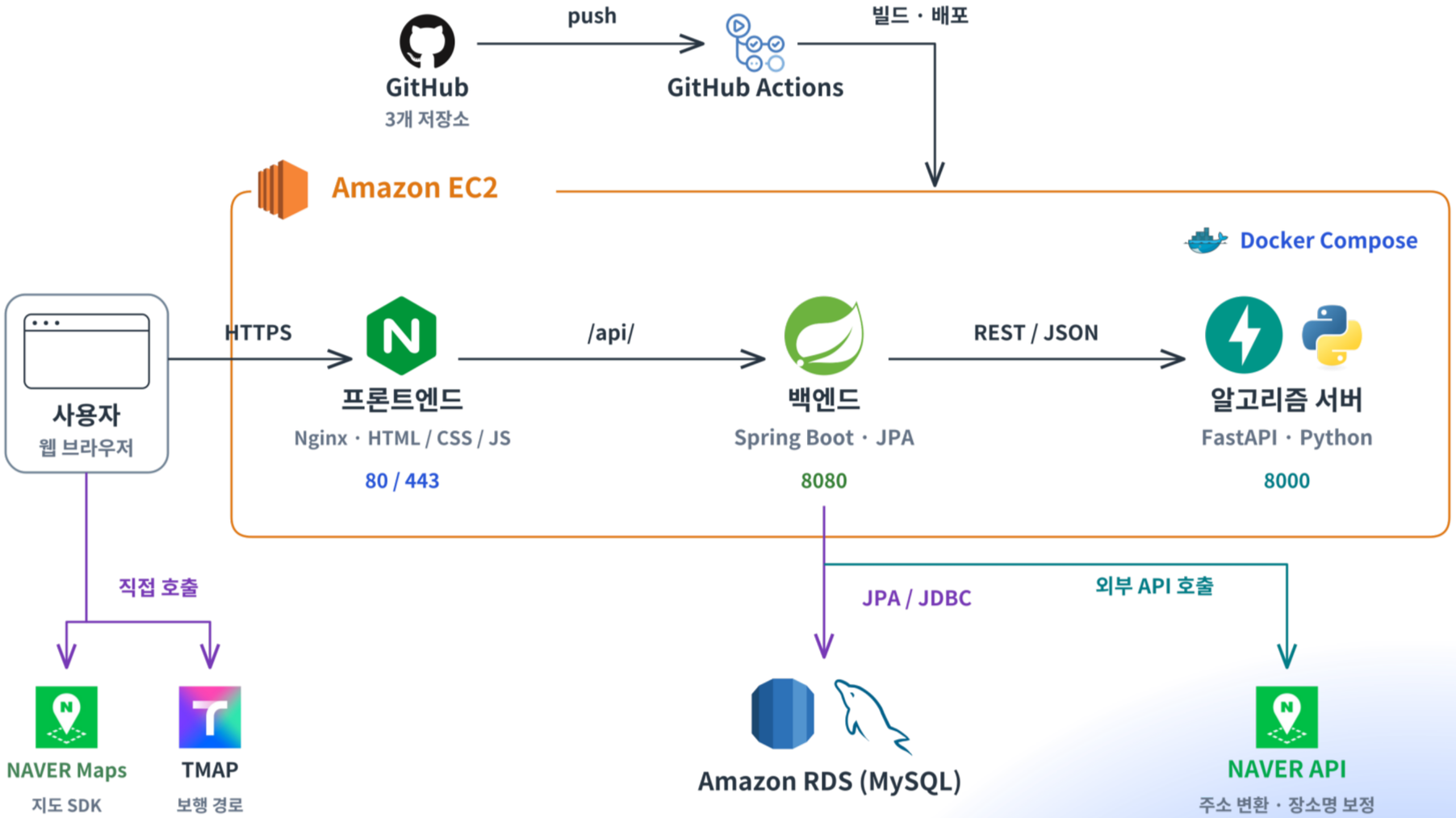
ALGORITHM

SCM

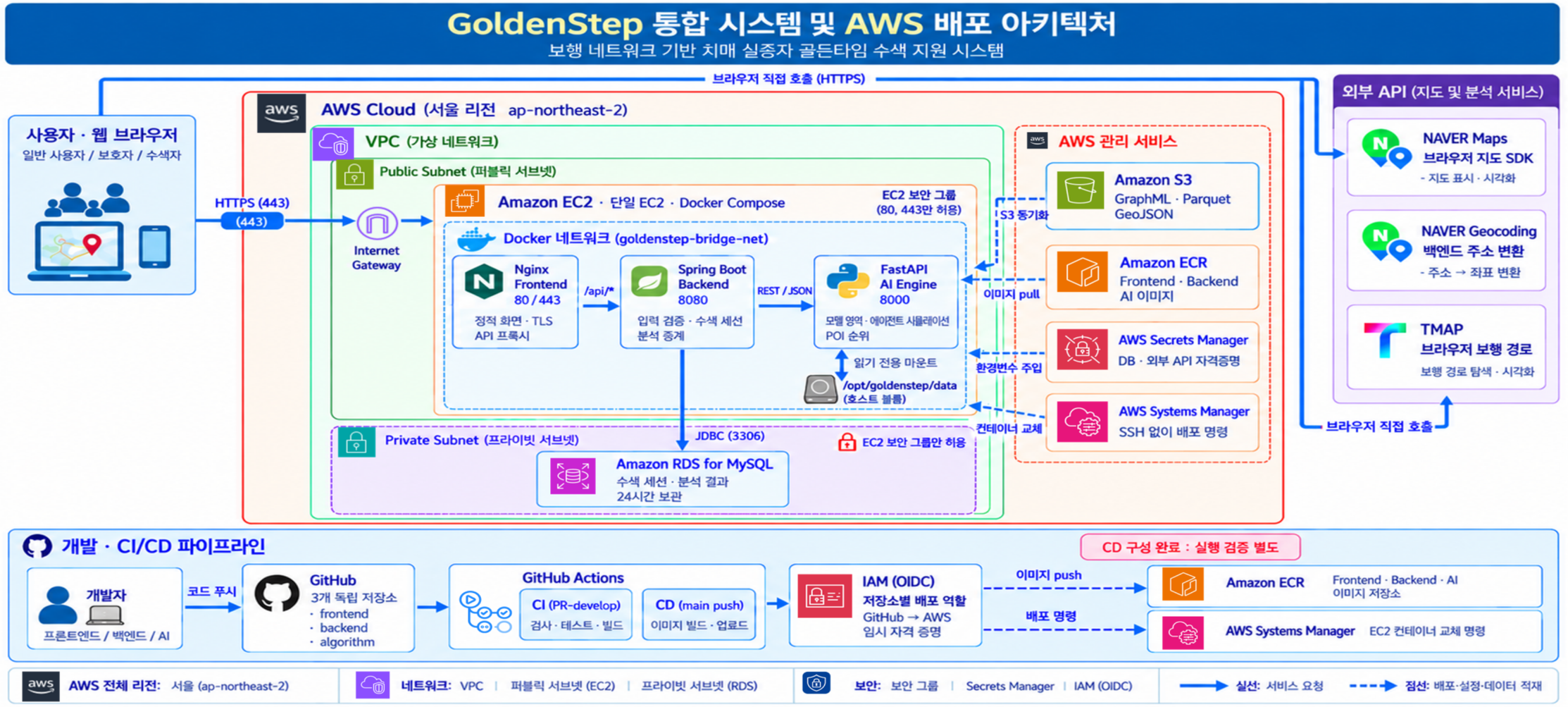
Front
HTML 5
CSS 3
JavaScript ES6+
Nginx 1.27-alpine
Back
Java 21
SpringBoot 4.1.1
Spring data JPA 4.1.1
lombok 1.18.46
MySQL 8.4

Algorithm
Python 3.11
Fast API 0.100.0
Numpy 1.24.0
NetworkX 3.0
Shapely 2.0.0
GeoPandas 0.13.0
OSMnx 1.6.0
SCM
Git
GitHub

1.4. 프로젝트 구조 구성도



1.4. 프로젝트 구조 배포 아키텍처



1.5. 활용방안 및 기대 효과

실종 사건 발생 초기에는 제한된 시간과 인력으로 수색을 진행해야 하므로, **신속하게 수색 범위를 설정하고 우선 확인 장소를 정하는 것**이 중요

실종자의 마지막 확인 위치와 시각, 대상자 정보를 바탕으로 **예상 이동 범위와 우선 확인 장소를 제공**하여 보호자와 수색 인력의 초기 수색 판단 지원

경찰·소방·지자체 등 공공기관의 실종 대응 과정에서 **수색 계획 수립을 위한 참고 자료로 활용**하고, 시간 경과에 따른 예측 결과를 통해 수색 범위 조정 지원

지도 기반 수색 결과와 장소 확인 상태를 공유하여 **수색 참여자 간 정보 전달을 돕고**, 중복 확인을 줄여 제한된 수색 인력의 효율적인 배치에 기여

아동·고령자 등 실종 시 위험에 노출되기 쉬운 대상자의 수색을 지원하여 **지역사회 안전망 강화와 실종자 조기 발견에 기여할 것으로 기대**

02

프로젝트 팀 구성 및 역할

2. 프로젝트 팀 구성 및 역할

	이재훈 (팀장)	김동규	박철우
구성	Backend	Geo Algorithm	Frontend
역할	• 탐색 정보 및 복구용 토큰 저장 • Naver 검색 API 기반 지명 검색 • 만료된 탐색 세션 배치 자동 삭제 • 시간대별 분석 결과 저장 • 공유 스냅샷 세션 및 복구용 토큰 저장	• 공간 데이터 사전 연산 및 로컬 캐싱 • 기준 보행 속도 기반 도달 권역 엔진 모듈 구현 • 터널 시야, Tobler 경사 저항, 도로 위계를 결합한 다항 로짓 효용 점수식 및 2,000명 Monte Carlo 경로 추적 시뮬레이터 구축 • 건축물과 공공 거점을 STRtree로 색인 후 TOP 3 선별(POI 랭킹) • REST API 엔드포인트, Lifespan 사전 적재 및 스키마 설계 • Koester ISRID 실측 통계 대조 K-S 검정	• 메인 화면 및 안전 공지 모달 • 탐색정보 입력 기능 • Naver Maps API 기반 마지막 확인 위치 검색/선택 • 분석 진행 상태 화면 • 우선확인지역 및 탐색결과 조회 • 시간대별 탐색 슬라이더 • 지도 기반 탐색결과 표시 • 탐색보드 정보 시각화 • 탐색정보 공유 스냅샷

03

프로젝트 수행 절차 및 방법

3.1. 수행 절차 도식화

3.2. WBS

3.1. 수행 절차 도식화

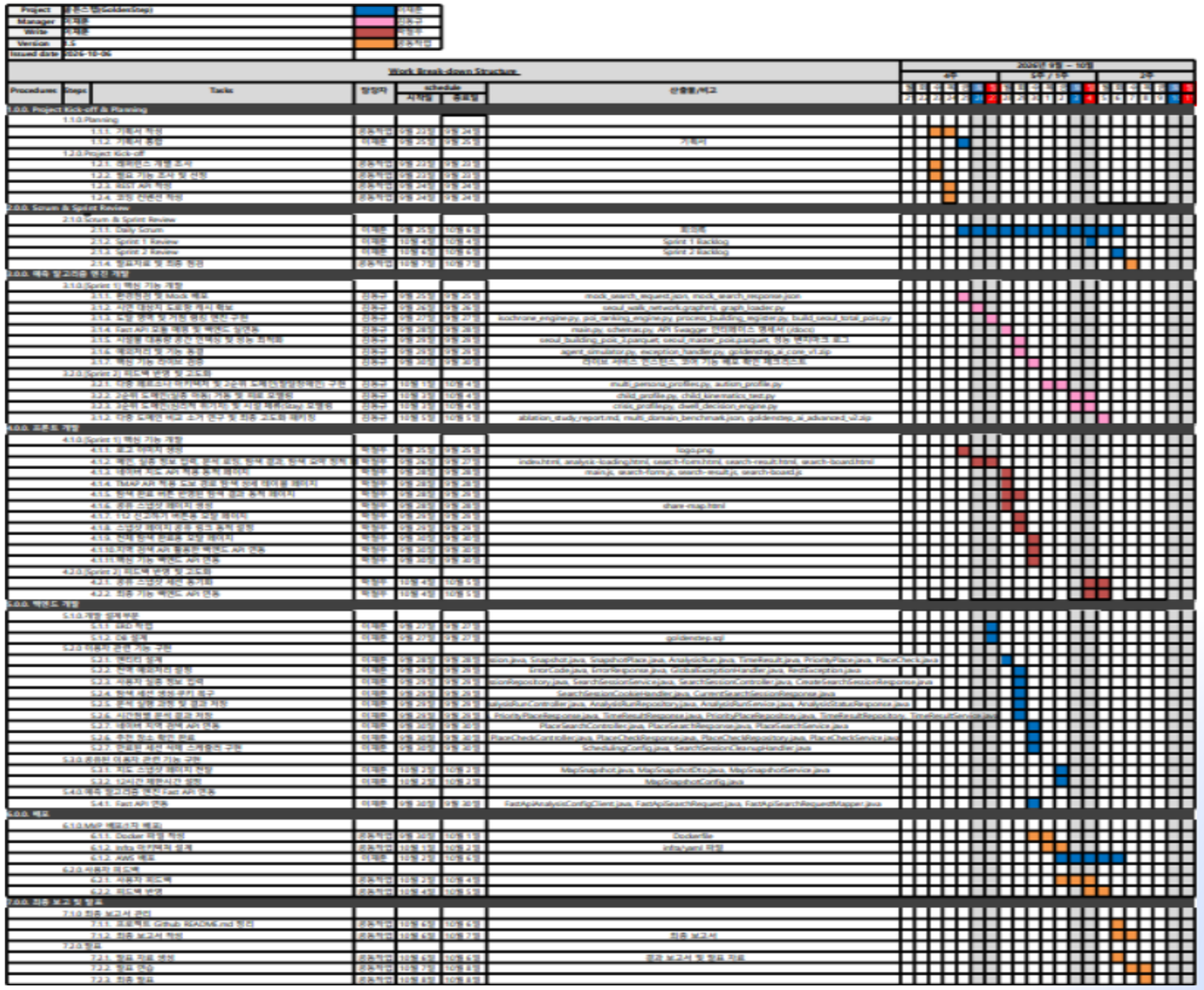
기획
개발
통합 및 배포
피드백
최종 배포
요구사항 및 기능 정의
WBS 작성
사이트맵 및 화면 설계
API 및 개발 규칙 정의
Frontend 개발
Backend 개발
AI 분석 엔진 개발
지도 API 연동
Front · Back · AI 연동
Docker 환경 구성
CI/CD 구성
AWS 1차 배포
사용자 테스트
설문조사 진행
사용자 피드백 수집
피드백 반영
기능/UI 개선
AWS 최종 배포
최종 검증

3.2.1. WBS 간략화

구분	주요 항목	담당	일정	주요 산출물
착수 및 프로젝트 관리	프로젝트 기획 및 일정 관리	팀 전체	09/23 ~ 10/08	기획서, WBS
분석 / 설계	레퍼런스 조사, 기능 선정, REST API 설계, 코딩 컨벤션 정의		09/23 ~ 09/25	기능 정의서, API 명세, 개발 규칙
AI 엔진 개발	도로망 데이터 구축, 도달영역·거점 랭킹, FastAPI 연동 및 검증	김동규	09/25 ~ 10/05	분석 엔진, FastAPI
프론트엔드 개발	화면 구현, 지도 API 연동, 동적 기능 및 Backend API 연동	박철우	09/26 ~ 10/02	웹 화면, 지도 기능
백엔드 개발	DB/Entity 설계, 탐색 세션·분석 결과·공유 API 개발, FastAPI 연동	이재훈(팀장)	WBS 기준 일정	Backend API, DB
배포	Docker 구성, 인프라 설계, AWS 배포, 사용자 피드백 반영	이재훈(팀장)	WBS 기준 일정	Docker, AWS 배포

3.2.2. WBS 상세

wbs 원문



04

프로젝트 수행 경과

4.1. 탐색정보 입력 기능

4.2. 알고리즘 기반 분석 기능

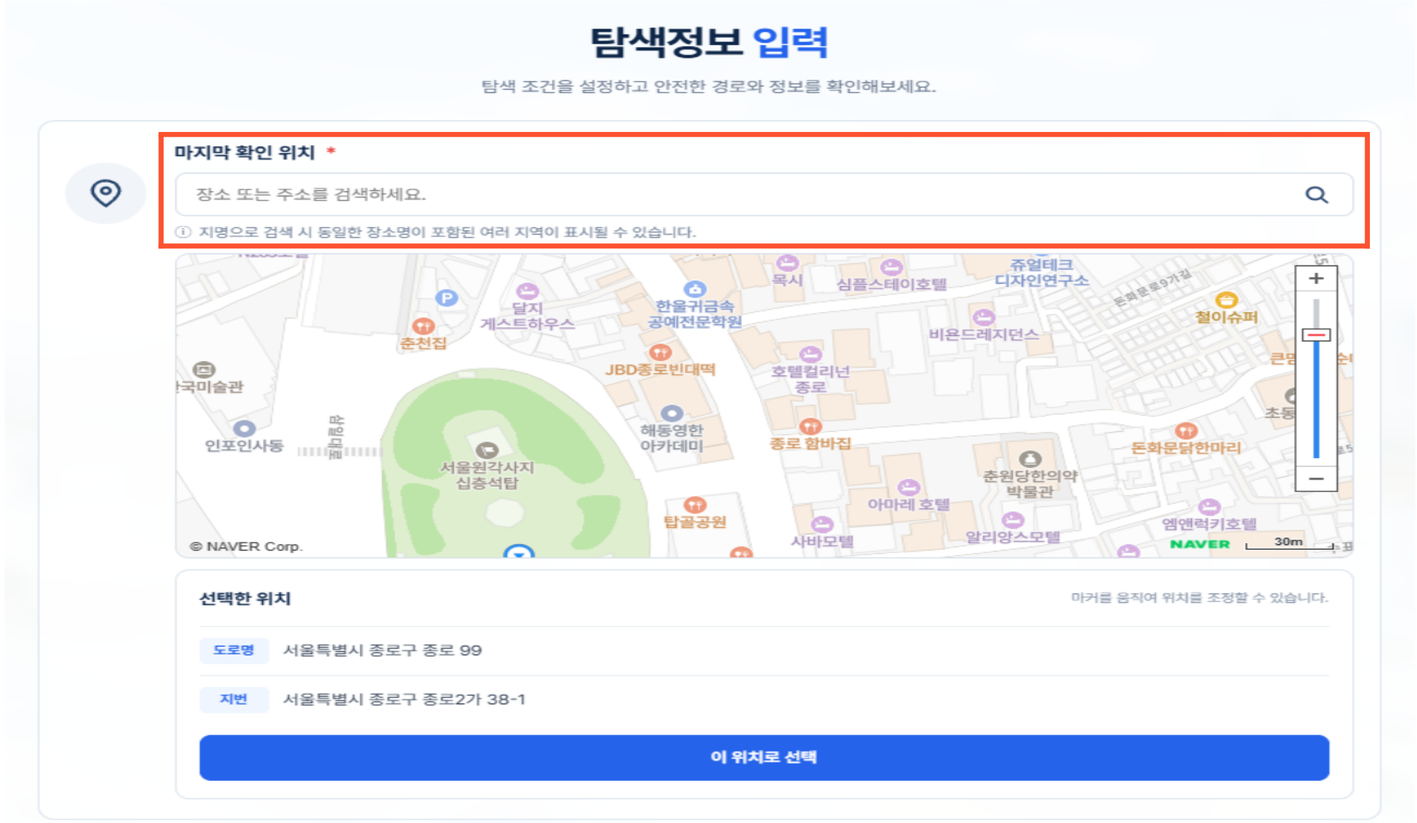
4.3. 시간대별 결과 확인 기능

4.4. 탐색 완료 기능

4.5. 공유 스냅샷 기능

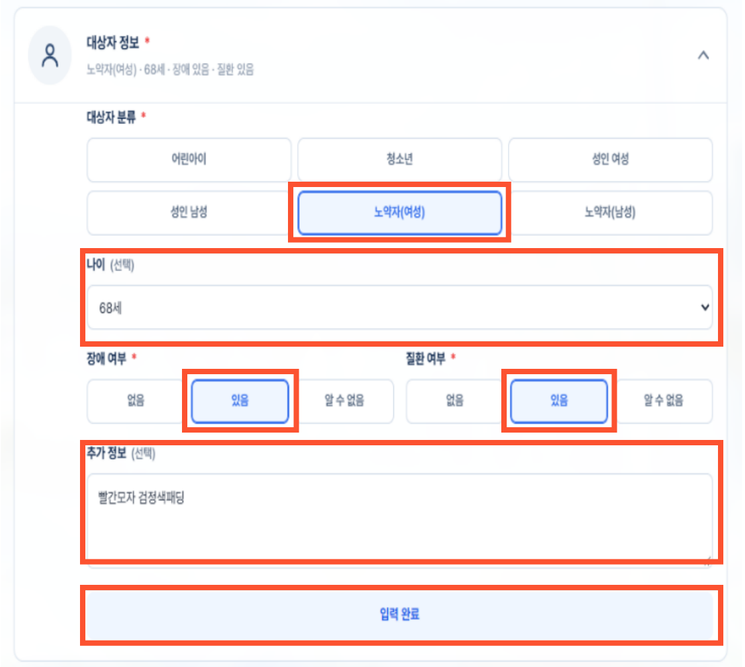
4.1
탐색정보 입력 기능

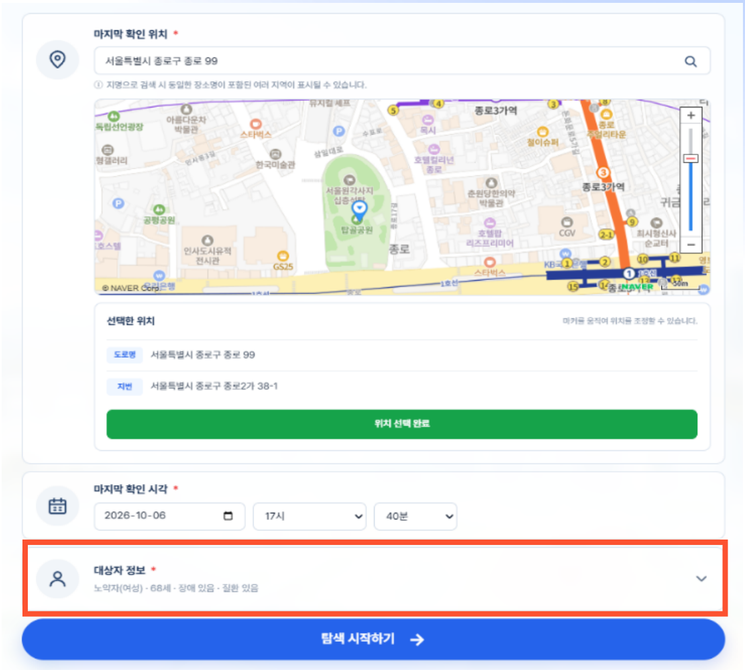
4.1. 탐색정보 입력 기능

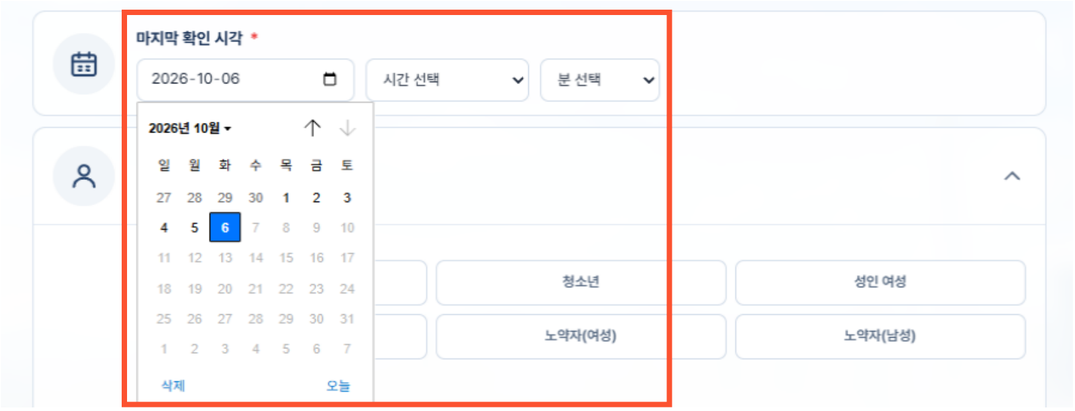


- Naver Maps API 기반 장소 검색 기능 구현
- 검색 결과 선택 또는 지도 클릭을 통한 위치 지정
- 선택 좌표를 주소로 변환하여 마지막 확인 위치 표시
- 선택된 위치의 위도·경도 정보를 탐색 요청에 활용

4.1. 탐색정보 입력 기능







- 실종 시점을 정확하게 설정할 수 있도록 날짜 및 시간 선택 기능 제공
- 대상자의 나이·유형 등 주요 특성 정보 입력
- 장애·질환 및 기타 사항 등 추가 탐색정보 선택 입력
- 마지막 확인 위치를 장소 검색 또는 지도 선택을 통해 설정
- 입력한 정보를 기반으로 분석을 요청하고 AI 분석 진행 화면으로 이동

4.1. 탐색정보 입력 기능 Back

```java
@PostMapping("/api/search/input")
@ResponseStatus(HttpStatus.CREATED)
public CreateSearchSessionResponse create(
        @Valid @RequestBody CreateSearchSessionRequest request,
        HttpServletResponse response
) {
    CreateResult result = searchSessionService.create(request);

    cookieHandler.writeRecoveryCookie(
            response,
            result.getRecoveryToken()
    );

    return new CreateSearchSessionResponse(
            result.getSessionId(),
            result.getRunId(),
            result.getStatus(),
            result.getExpiresAt()
    );
}
```

```java
public void writeRecoveryCookie(
        HttpServletResponse response,
        String recoveryToken
) {
    ResponseCookie cookie = ResponseCookie.from(
                    COOKIE_NAME,
                    recoveryToken
            )
            .httpOnly(true)
            .secure(secure)
            .sameSite("Lax")
            .path("/")
            .maxAge(COOKIE_MAX_AGE)
            .build();

    response.addHeader(HttpHeaders.SET_COOKIE, cookie.toString());
}
```

- front에서 POST 요청으로 json 형식의 요청 Body를 입력받음
- HttpServlet 응답값에 recoveryToken을 Header로 추가

4.1. 탐색정보 입력 기능 Back

```java
private String generateRecoveryToken() {
    byte[] randomBytes = new byte[32];
    SECURE_RANDOM.nextBytes(randomBytes);
    return HexFormat.of().formatHex(randomBytes);
}

private String sha256(String value) {
    try {
        MessageDigest digest = MessageDigest.getInstance("SHA-256");
        byte[] hash = digest.digest(value.getBytes(StandardCharsets.UTF_8));
        return HexFormat.of().formatHex(hash);
    } catch (NoSuchAlgorithmException exception) {
        throw new IllegalStateException("SHA-256 해시 기능을 사용할 수 없습니다.", exception);
    }
}
```

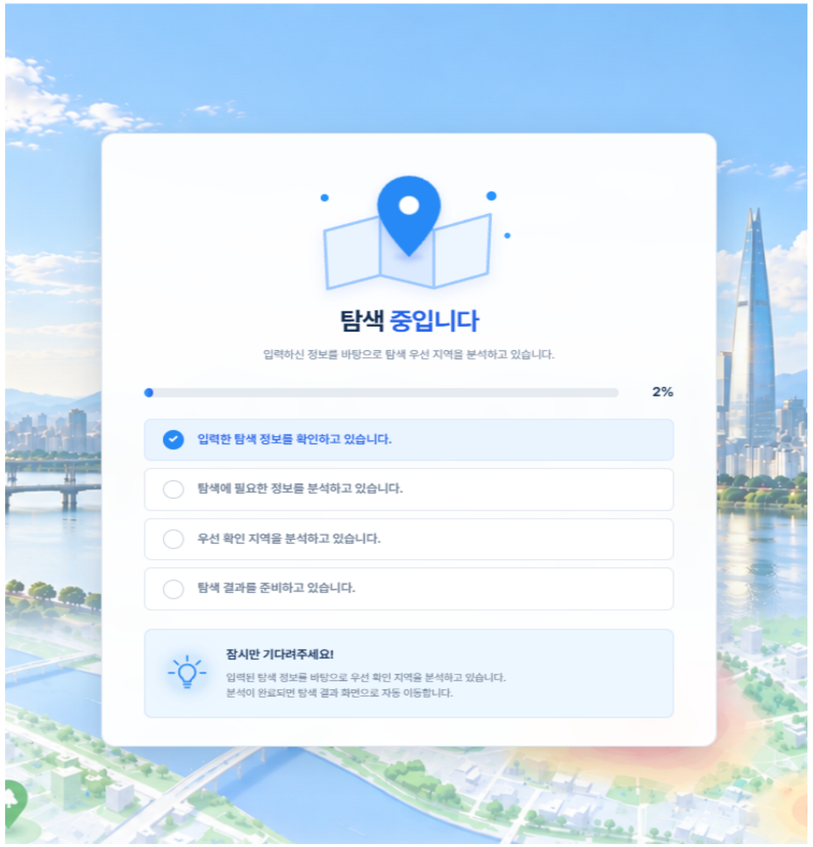
```java
public AnalysisRun(SearchSession searchSession, LocalDateTime requestedAt) {
    this.searchSession = searchSession;
    this.requestedAt = requestedAt;
    this.status = Status.PROCESSING;
}
```

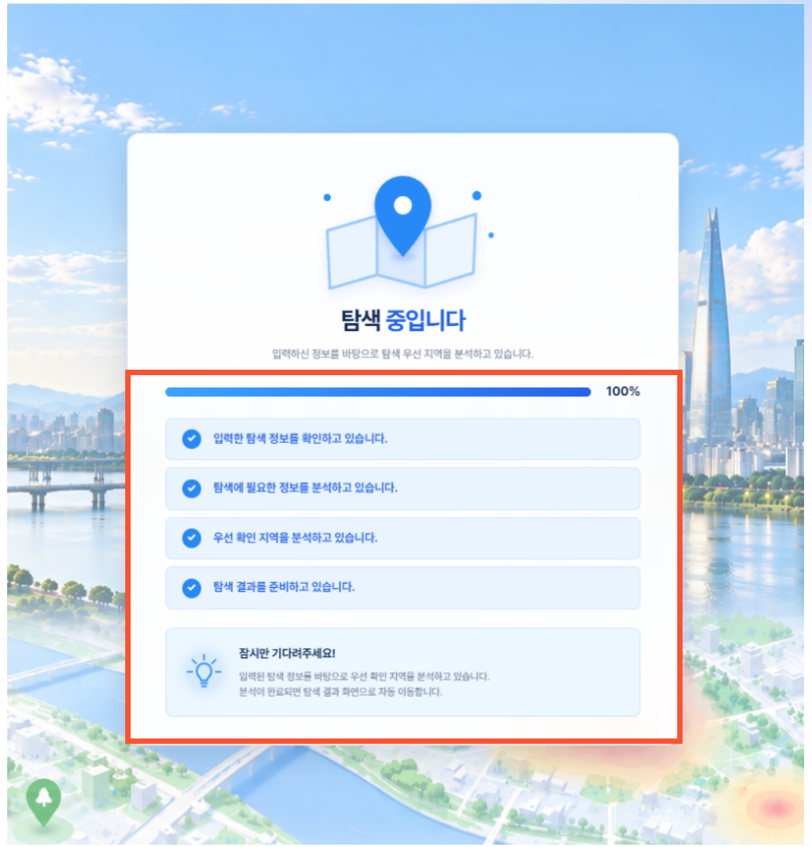
```java
@Transactional
public CreateResult create(CreateSearchSessionRequest request) {
    LocalDateTime now = LocalDateTime.now(SERVICE_ZONE).truncatedTo(ChronoUnit.SECONDS);

    validateLastSeenAt(request.getLastSeenAt(), now);

    String recoveryToken = generateRecoveryToken();
    String recoveryTokenHash = sha256(recoveryToken);

    SearchSession session = new SearchSession(
            recoveryTokenHash,
            request.getLastLat(),
            request.getLastLng(),
            normalizeOptionalText(request.getLastAddress()),
            request.getLastSeenAt(),
            createPersonAttributes(request),
            now
    );

    SearchSession savedSession = searchSessionRepository.save(session);

    AnalysisRun run = new AnalysisRun(savedSession, now);

    AnalysisRun savedRun = analysisRunRepository.save(run);

    eventPublisher.publishEvent(
            new AnalysisRequestedEvent(savedRun.getId())
    );

    return new CreateResult(
            savedSession.getId(),
            savedRun.getId(),
            savedRun.getStatus(),
            recoveryToken,
            savedSession.getExpiresAt()
    );
}
```

- 복구 토큰은 32byte의 난수를 생성 후 SHA-256으로 Hash한 값
- 생성자를 통해 SearchSession 객체 생성 후 JPA save() 실행
- 분석 상태를 나타내는 AnalysisRun 객체 생성 후 JPA save() 실행
- FastAPI로 분석 결과를 요청하는 event 등록

4.2
알고리즘 기반 분석 기능

4.2. AI 분석 진행 기능





- 입력된 탐색정보를 기반으로 AI 분석 요청
- 분석이 진행되는 동안 진행 상태와 진행률을 화면에 표시
- 분석 중 사용자가 확인할 수 있도록 탐색 관련 안내 정보 제공
- 분석 완료 여부를 확인하여 탐색결과 화면으로 자동 이동

4.2. 알고리즘 기반 분석 기능 Back

```java
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
public void handle(AnalysisRequestedEvent event) {
    Long runId = event.getRunId();

    try {
        analysisExecutor.execute(() -> executeAnalysis(runId));

    } catch (TaskRejectedException exception) {
        log.error(
                "분석 작업을 등록하지 못했습니다. "
                + "실행기 포화 또는 종료 상태를 확인해주세요. runId={}",
                runId,
                exception
        );

        markFailed(runId);
    }
}
```

```java
@Configuration
public class AsyncConfig {

    @Bean(name = "analysisExecutor")
    ThreadPoolTaskExecutor analysisExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
```

```java
private void executeAnalysis(Long runId) {
    try {
        AnalysisPredictionRequest request = analysisRunService.createInitialPredictionRequest(runId);

        analysisRunService.executeInitialAnalysis(
                runId,
                request
        );
```

```java
@Transactional(propagation = Propagation.NOT_SUPPORTED)
public void executeInitialAnalysis(Long runId, AnalysisPredictionRequest request) {
    validatePredictionRequest(runId, request);

    if (request.getTimePoint() != TimePoint.NOW) {
        throw new RestException(ErrorCode.INVALID_INPUT);
    }

    executePrediction(runId, request);
}
```

```java
@Transactional(propagation = Propagation.NOT_SUPPORTED)
public Long executePrediction(Long runId, AnalysisPredictionRequest request) {
    validatePredictionRequest(runId, request);

    AnalysisPredictionResponse prediction = analysisClient.analyze(request);
```

- SearchSession과 AnaysisRun이 DB에 commit되는 트랜잭션 이후에 AnalysisRequestEvent의 handle() 실행
- Fast API 분석 결과를 기다리는 것과 /api/search/input 요청은 비동기로 실행
- createInitialPredictionRequest()에서 현재 세션이 PROCESSING인지 확인후 executeInitialAnalysis()실행
- 최종적으로 FastAPI 연동을 담당하는 analysisClient.analyze()를 호출

4.2. 알고리즘 기반 분석 기능 Back

```java
@Override
public AnalysisPredictionResponse analyze(
        AnalysisPredictionRequest request
) {
    FastApiSearchRequest apiRequest = requestMapper.map(request);
```

```json
{
  "request_id": "req_20260925_001",
  "missing_person": {
    "person_type": "DEMENTIA",
    "last_seen_location": {
      "lat": 37.566535,
      "lon": 126.977969,
      "address": "서울특별시 중구 태평로1가 31 (서울시청 앞)"
    },
    "last_seen_time": "2026-09-25T14:30:00+09:00",
    "elapsed_hours": 1.5
  },
  "parameters": {
    "num_agents": 2000,
    "search_radius_m": 3500.0,
    "fallback_level": "PLAN_A"
  }
}
```

```java
ResponseEntity<String> httpResponse = restClient.post()
        .uri(SEARCH_PATH)
        .contentType(MediaType.APPLICATION_JSON)
        .accept(MediaType.APPLICATION_JSON)
        .body(requestBody)
        .retrieve()
        .toEntity(String.class);

String responseBody = httpResponse.getBody();

AnalysisPredictionResponse response =
        jsonMapper.readValue(
                responseBody,
                AnalysisPredictionResponse.class
        );
```

```java
public Long executePrediction(Long runId, AnalysisPredictionRequest request)
return timeResultService.saveResult(
        runId,
        request.getTimePoint(),
        request.getTargetAt(),
        prediction.getBoundaryZone(),
        prediction.getSummary().getReliabilityStatus(),
        places
);
```

- FastAPIAnalysisClient에서 analyze() 실행
- 프론트로부터 받아 저장한 요청값을 requestMapper에서 FastAPI 요청 형식에 맞게 변환 후 POST 요청
- 결과를 받아 다시 AnalysisRunService.executePrediction()로 돌아옴
- 결과를 DB에 저장하는 timResultService.saveResult() 호출

4.2. 알고리즘 기반 분석 기능 Back

```java
@GetMapping("/api/search/analysis/{runId}")
public AnalysisStatusResponse getStatus(
        @PathVariable Long runId,
        @CookieValue(name = "GOLDENSTEP_RECOVERY", required = false) String recoveryToken
) {
    return analysisRunService.getStatus(runId, recoveryToken);
}
```

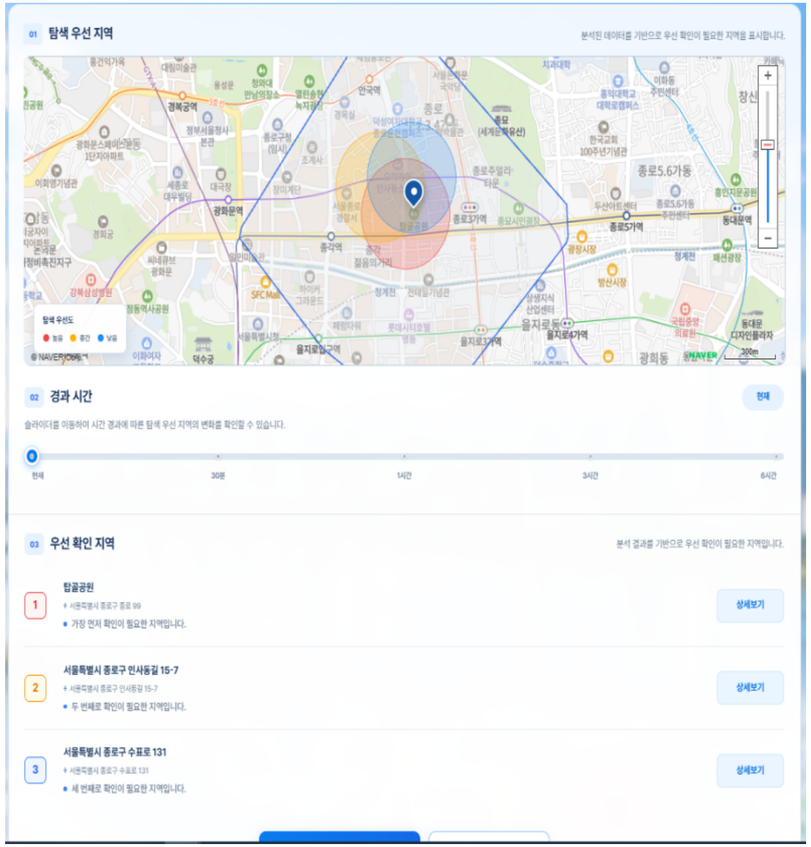
```java
@Transactional(readOnly = true)
public AnalysisStatusResponse getStatus(Long runId, String recoveryToken) {
    SearchSession session = searchSessionService.restore(recoveryToken);

    AnalysisRun run = analysisRunRepository.findByIdAndSearchSession(runId, session)
            .orElseThrow(() -> new RestException(ErrorCode.ANALYSIS_RUN_NOT_FOUND));

    return new AnalysisStatusResponse(run);
}
```

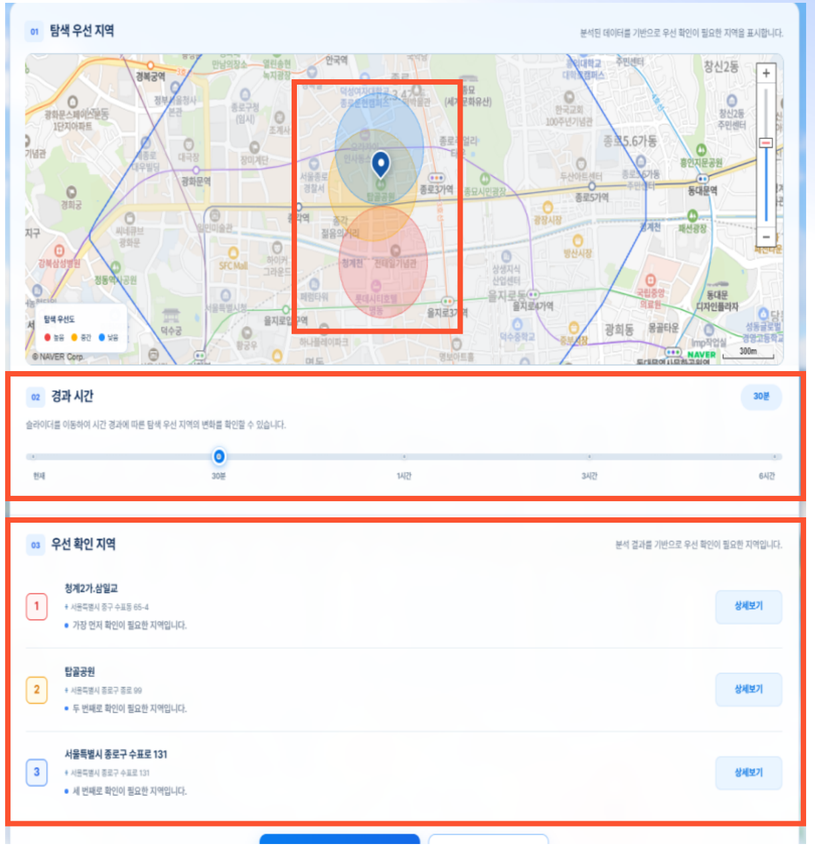
```java
public SearchSession restore(String recoveryToken) {
    if (recoveryToken == null || recoveryToken.isBlank()) {
        throw new RestException(ErrorCode.RECOVERY_TOKEN_MISSING);
    }

    String tokenHash = sha256(recoveryToken);

    SearchSession session = searchSessionRepository
            .findByRecoveryTokenHash(tokenHash)
            .orElseThrow(() ->
                    new RestException(ErrorCode.SEARCH_SESSION_NOT_FOUND)
            );

    LocalDateTime now = LocalDateTime.now(SERVICE_ZONE);

    if (!session.getExpiresAt().isAfter(now)) {
        throw new RestException(ErrorCode.SEARCH_SESSION_EXPIRED);
    }

    return session;
}
```

- front에서 GET 요청으로 분석 ID {runId}를 입력 받음
- 헤더에 있는 recoveryToken을 SearchSessionService의 restore(String recoveryToken) 실행
- 입력 받은 Token 문자열을 Hash하여 DB에 있는 Token과 비교하여 있으면 세션 반환, 없으면 예외 발생
- 해당 세션의 id를 외래키로 갖는 시간분석 데이터를 갖는지 확인, 없으면 예외 발생

4.3.
시간대별 결과 확인 기능

4.3. 시간대별 결과 확인 기능

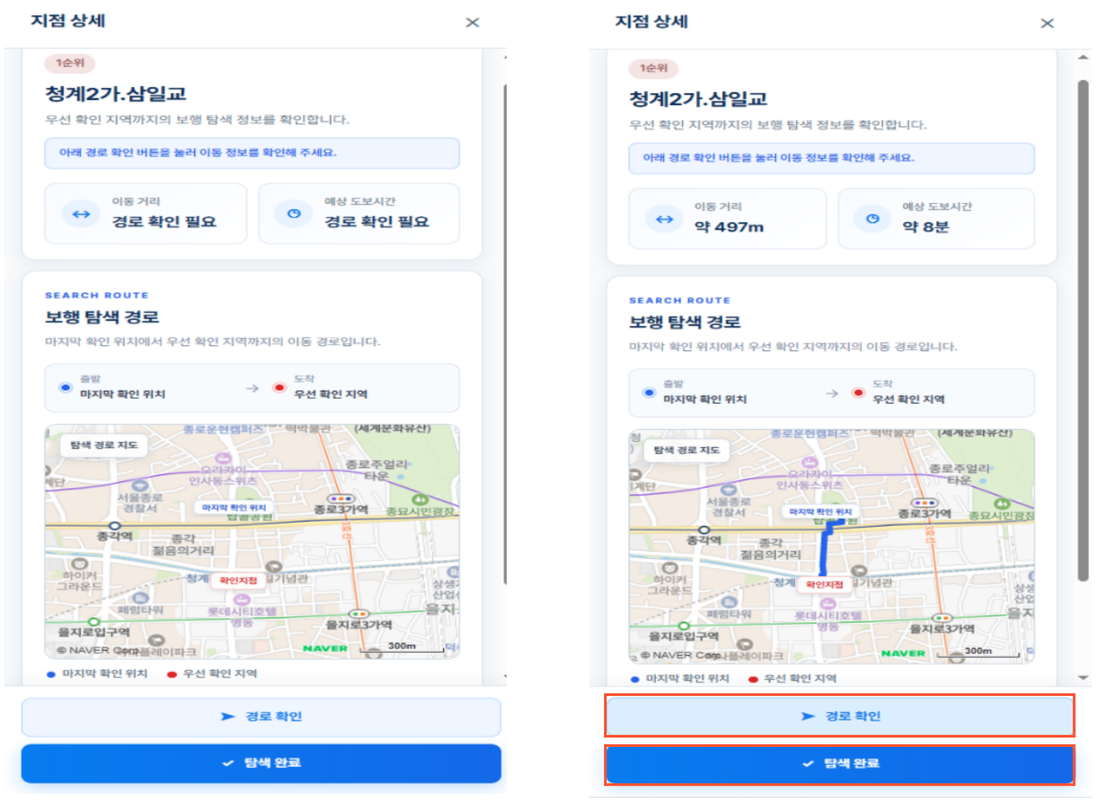


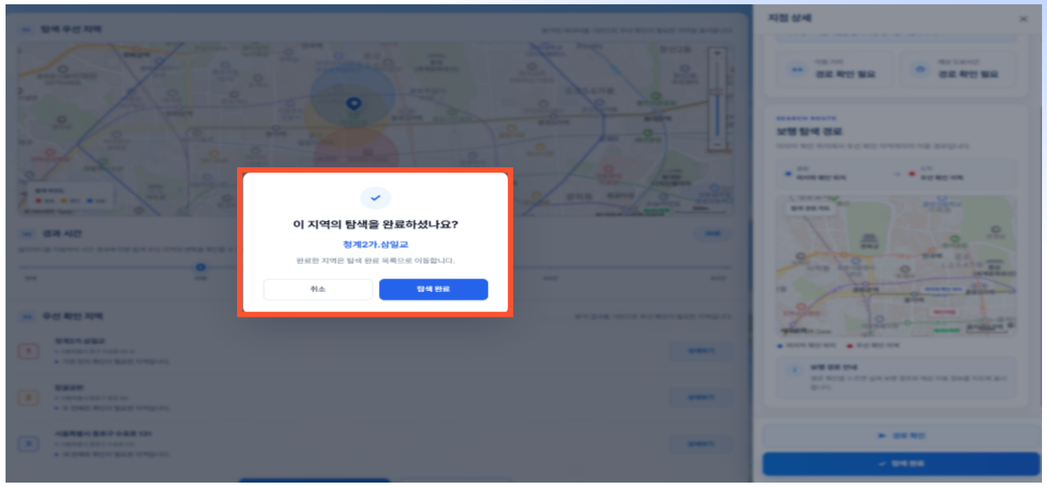


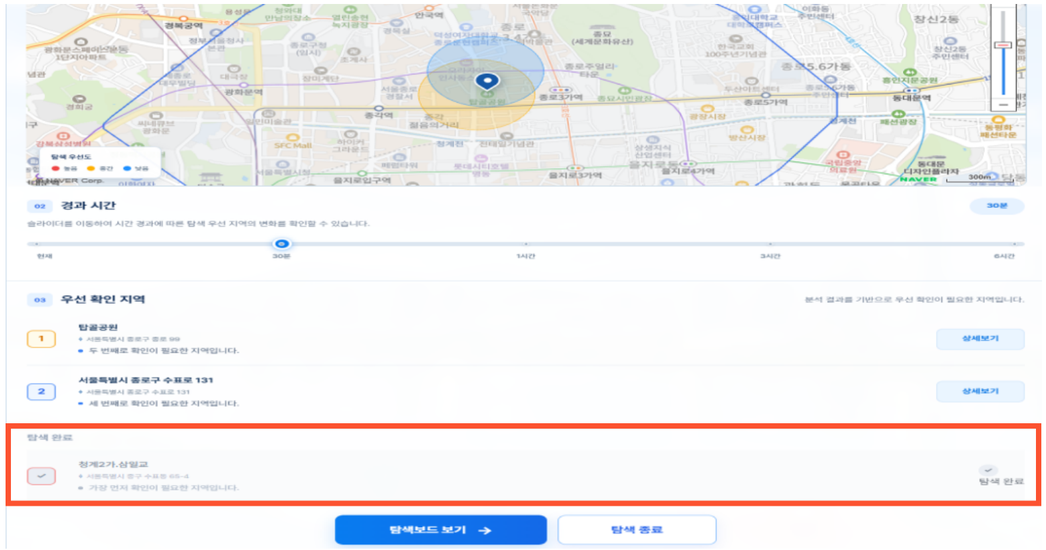
- 현재·30분·1시간·3시간·6시간 기준으로 시간대별 탐색결과 확인
- 시간 슬라이더를 조작하여 경과시간에 따른 분석 결과 조회
- 선택한 시간에 따라 지도 내 우선확인지역과 탐색 범위를 시각적으로 표시
- 시간대별 결과를 비교하여 탐색 범위의 변화와 우선확인지역을 직관적으로 확인

4.4.
탐색 완료 기능

4.4. 탐색 완료 기능



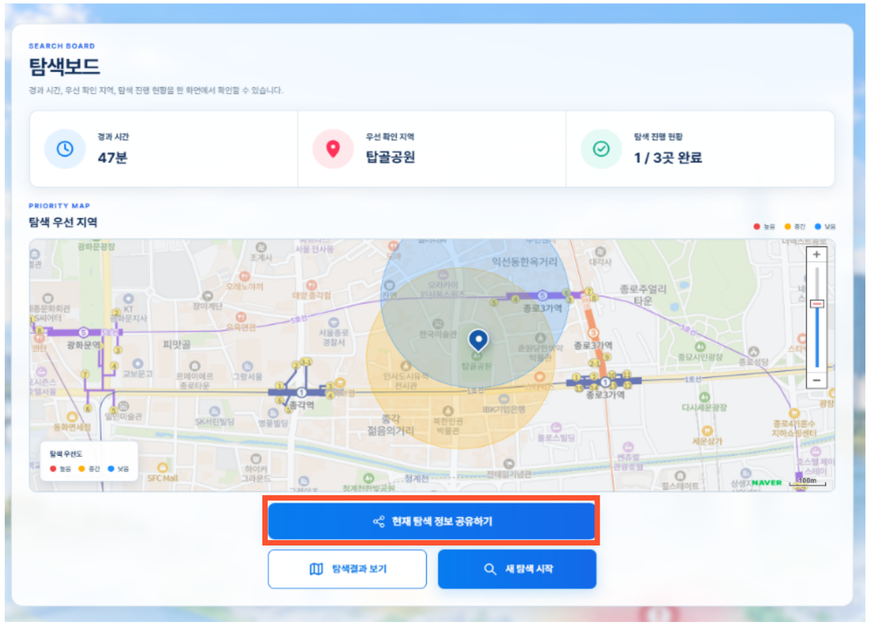




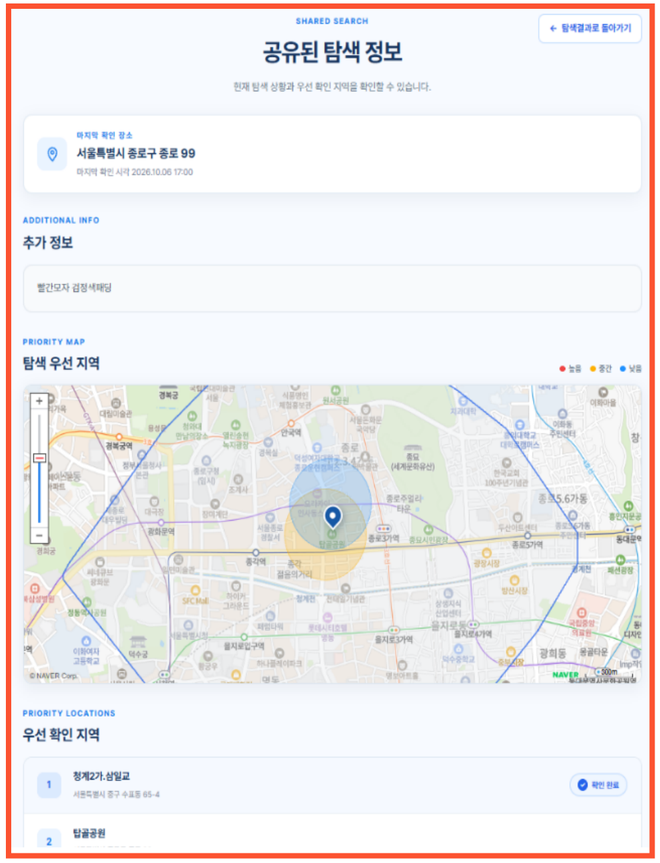
- 상세보기 기능
- 우선확인지역 목록에서 원하는 지역을 선택하여 상세정보 확인
- 선택한 지역의 우선도, 거리 및 탐색 관련 정보 제공
- 지도와 상세정보를 함께 제공하여 우선확인지역의 위치와 특징을 직관적으로 확인
- 탐색 종료 시 탐색완료 기능을 통해 현재 탐색을 종료
- 완료 여부를 다시 확인할 수 있도록 확인 모달 제공
- 탐색완료 후 탐색 결과를 정리하여 탐색보드 화면으로 이동

4.5.
공유 스냅샷 기능

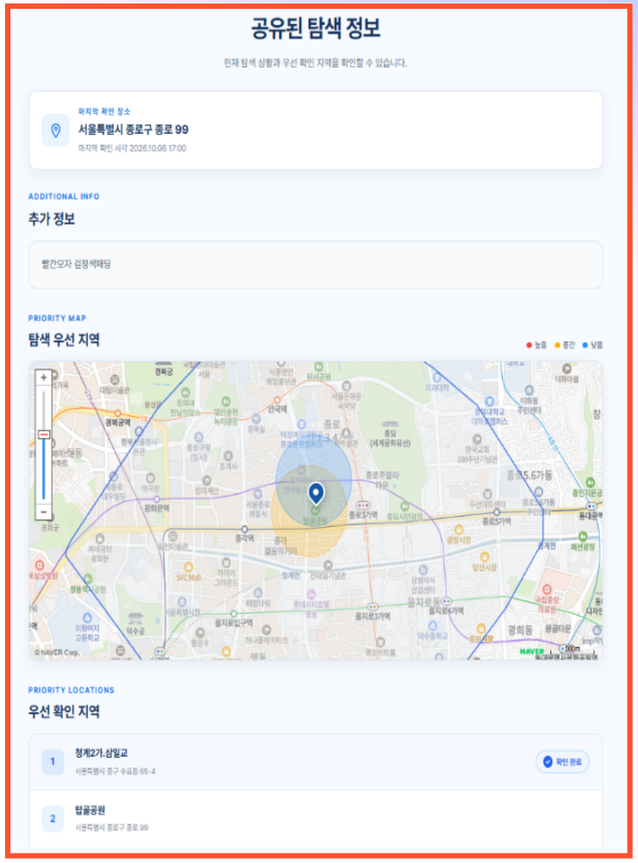
4.5. 공유 스냅샷 기능



사용자 화면



공유받은 사용자 화면



- 공유 스냅샷 기능
- 현재 탐색 결과를 공유 스냅샷 형태로 생성하여 공유
- 공유 시점의 탐색정보, 지도 및 우선확인지역 정보를 함께 제공
- 공유받은 사용자가 별도의 입력 과정 없이 동일한 탐색 결과를 확인
- 공유된 탐색정보는 일정 시간이 지나면 만료되도록 구성

05

피드백 반영

5.1. 사용자 피드백

5.1. 사용자 피드백

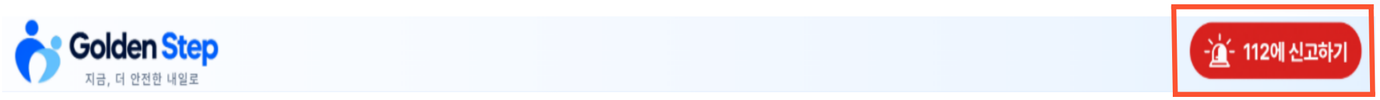
사용자	피드백	답변 및 개선사항
구글 폼 피드백(익명)	긴급 상황에서 사용하는 112 신고 버튼이 눈에 잘 띄지 않아 바로 찾기 어렵다.	버튼의 색상과 시각적 강조를 개선하여 긴급 신고 기능을 빠르게 인지할 수 있도록 수정
구글 폼 피드백(익명)	탐색정보 입력 화면에서 대상자 정보 입력 영역을 바로 확인하기 어렵다.	최초 진입 시 대상자 정보 영역이 펼쳐진 상태로 표시되도록 변경하여 입력 흐름 개선
구글 폼 피드백(익명)	위치를 검색했을 때 같은 장소명이 여러 개 표시되어 어떤 위치를 선택해야 하는지 헷갈릴 수 있다.	위치 검색 특성을 사용자가 미리 인지할 수 있도록 검색 영역에 “지명으로 검색 시 동일한 장소명이 포함된 여러 지역이 표시될 수 있습니다.” 안내 문구 추가
구글 폼 피드백(익명)	공유받은 화면이 공유한 사용자가 보고 있던 탐색 결과 화면과 달라 정보 확인이 어렵다.	공유 시점의 탐색정보와 결과가 동일하게 전달되도록 공유 스냅샷의 데이터 및 화면 동기화 개선

5.1.1. 112 신고 버튼 시인성 개선

- 피드백

  긴급 상황에서 사용하는 112 신고 버튼이
  눈에 잘 띄지 않아 바로 찾기 어렵다.

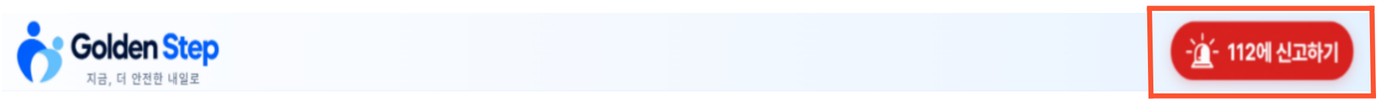
수정전



- 개선

  버튼의 색상과 시각적 강조를 개선하여
  긴급 신고 기능을 빠르게 인지할 수 있도록 수정

수정후



5.1.2. 대상자 정보 입력 편의성 개선

- 피드백

  탐색정보 입력 화면에서 대상자 정보
  입력 영역을 바로 확인하기 어렵다.

- 개선

  최초 진입 시 대상자 정보 영역이 펼쳐진 상태로
  표시되도록 변경하여 입력 흐름 개선

5.1.3. 마지막 확인 위치 검색 안내 추가

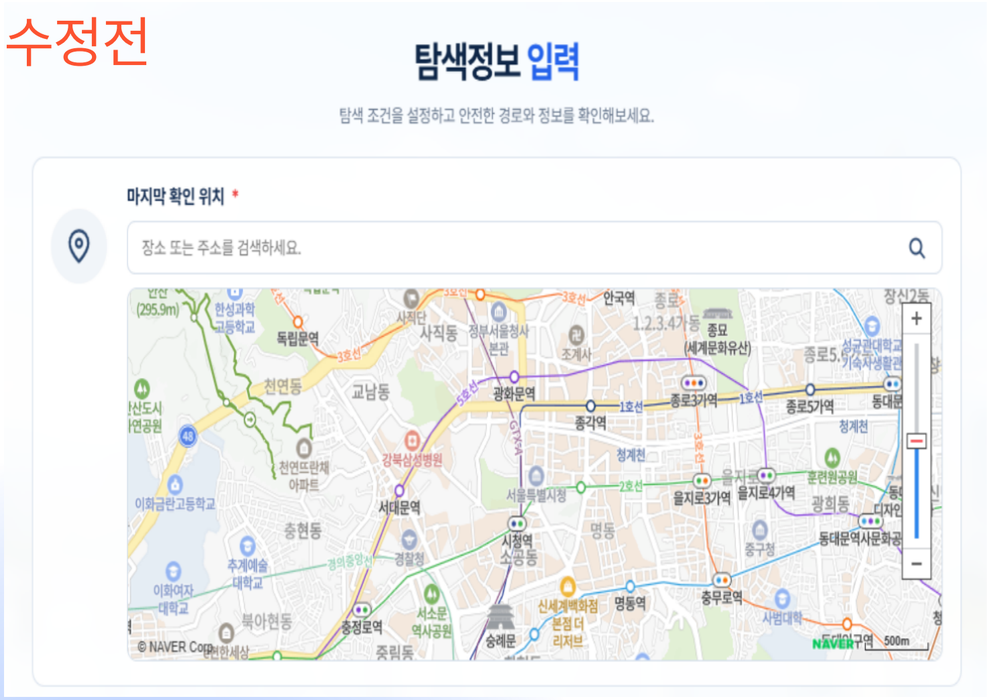
- 피드백

  위치를 검색했을 때 같은 장소명이 여러 개 표시되어 어떤 위치를 선택해야 하는지 헷갈릴 수 있다.

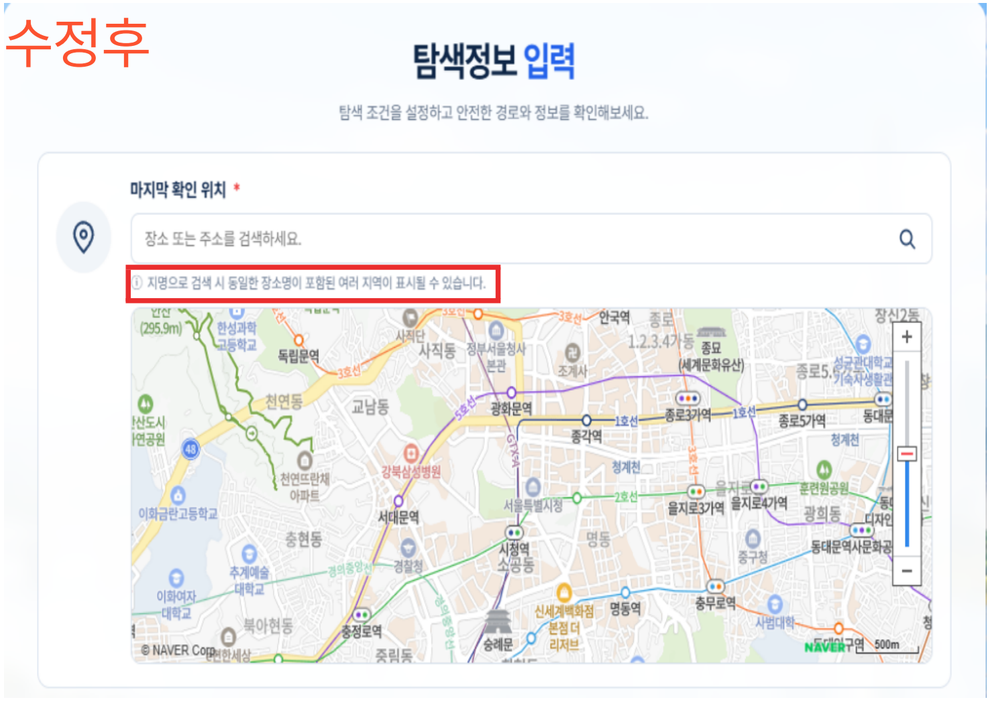
- 개선

  위치 검색 특성을 사용자가 미리 인지할 수 있도록 검색 영역에 “지명으로 검색 시 동일한 장소명이 포함된 여러 지역이 표시될 수 있습니다.” 안내 문구 추가

수정전



수정후



5.1.4. 공유 스냅샷 화면 일치성 개선

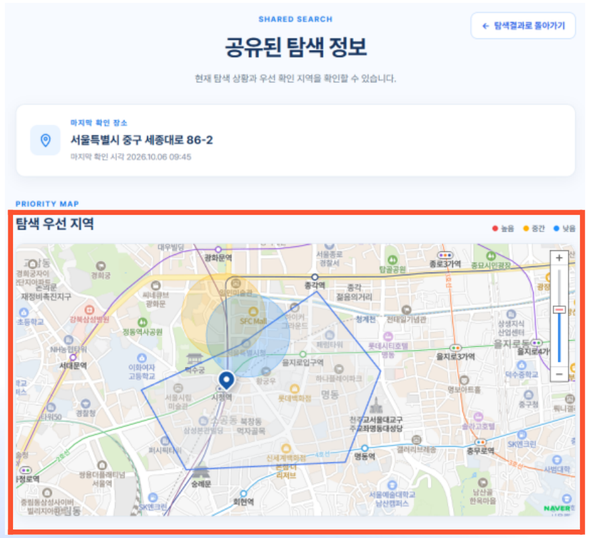
- 피드백

  공유받은 화면이 공유한 사용자가 보고 있던
  탐색 결과 화면과 달라 정보 확인이 어렵다.

- 개선

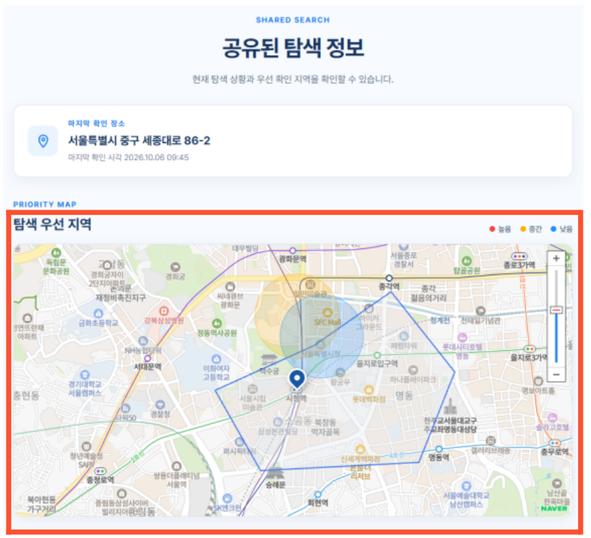
  공유 시점의 탐색정보와 결과가 동일하게 전달
  되도록 공유 스냅샷의 데이터 및 화면 동기화 개선

사용자가 공유한 화면



공유받은 사용자의 화면 (수정전)

공유받은 사용자의 화면 (수정후)



06

트러블 슈팅

6.1. Naver Maps API Key 관리 및 환경 설정 개선

6.2. 추천 장소 식별 코드 데이터 가공

6.3. 대규모 공간 연산 병목 제거

6.1. Naver Maps API Key 관리 및 환경 설정 개선

문제

Naver Maps API Client ID를 프론트엔드 코드에서 직접 관리하면서 Key 노출 위험과
개발·배포 환경별 설정 관리에 어려움이 발생했습니다.

API Key를 별도 환경 설정값으로 분리

```
window.ENV = {
  NAVER_MAP_CLIENT_ID: "실제 NAVER ID",
  TMAP_APP_KEY: "실제 APP KEY 값"
};

window.ENV = {
  NAVER_MAP_CLIENT_ID: "실제 NAVER ID",
  TMAP_APP_KEY: "실제 APP KEY 값",
  API_BASE_URL: "http://localhost:8081"
};
```

원인

정적 프론트엔드 환경에서는 서버처럼 환경변수를 직접 참조하기 어려워 API Key를
별도로 관리하고 전달할 구조가 필요했습니다. 또한 API Key가 소스 코드에 직접 노출될 경우 외부에서 무단으로 사용될
위험이 있어 보안 측면에서도 분리 관리가 필요했습니다.

해결 과정

API 설정값을 별도의 env-config.js로 분리했습니다. naver-map-loader.js에서 설정된
Client ID를 불러오고 Naver Maps API 스크립트를 동적으로 생성하도록 변경했습니다.

Client ID를 적용하여 Naver Maps API 동적 로드

```
(function () {
  const clientId = window.ENV?.NAVER_MAP_CLIENT_ID;

  window.naverMapsReady = new Promise((resolve, reject) => {
    if (!clientId) {
      reject(new Error("NAVER_MAP_CLIENT_ID가 설정되지 않았습니다."));
      return;
    }

    if (window.naver?.maps) {
      resolve(window.naver);
      return;
    }

    const script = document.createElement("script");
    script.src = `https://oapi.map.naver.com/openapi/v3/maps.js?ncpKeyId=${encodeURIComponent(clientId)}&submodules=geocoder`;

    script.onload = () => {
      resolve(window.naver);
    };

    script.onerror = () => {
      reject(new Error("NAVER Maps API를 불러오지 못했습니다."));
    };
```

6.2. 추천 장소 식별 코드 데이터 가공

문제

추천 장소명이 없거나 BLDG-숫자, OSM-숫자와 같은 식별자로 저장됨

poi_id	priority_rank	name	address
BLDG-344503	3	B0010000000BICVZE	서울특별시 서대문구 독립문로 12
OSM-357843903	1	신세계백화점	서울특별시 중구 소공로 65-1
BLDG-49258	2	1531 B00100000000T1QNL	서울특별시 중구 무교로 17-25

원인

FastAPI에서 좌표와 함께 장소명을 넘겨줄 때, 서울시 장소 데이터를 사용하여 전달. 만약 없을 경우 특정 식별자를 통해 그대로 전달함. 백엔드 내에서 변환 코드 부재

해결 과정

PlaceNameResolver를 만들어 장소명을 정규화 하는 코드를 새롭게 작성

1. 만약 FastAPI 응답값에 장소명이 있다면 그대로 사용
2. 식별자로 넘어오거나 장소명이 없는 경우, 좌표를 기준으로 Naver Maps API의 GeoCoding API 사용해 건물명 또는 장소명 조회
3. 2번 결과도 없는 경우 주소명 그대로 사용

```java
public boolean isUnknownName(String name) {
    if (name == null || name.isBlank()) {
        return true;
    }

    String normalized = name.strip().replaceAll("\\s+", " ");

    return UNKNOWN_NAMES.contains(
                normalized.toLowerCase(Locale.ROOT)
            )
            || BUILDING_CODE.matcher(normalized).matches()
            || POI_CODE.matcher(normalized).matches();
}
```

```java
        normalizedAddress,
        lat,
        lng
);

return buildingName == null
        ? normalizedAddress
        : buildingName;
```

6.3. 대규모 공간 연산 병목 제거

문제

서버 연산량 급증 및 타임아웃 장애

- 실종 경과 시간이 증가할수록 FastAPI AI 엔진의 연산 지연 및 Spring Boot 메인 서버의 응답 대기 시간 지연 장애 발생.

원인

- 화면 표출에 불필요한 수만 개 도로 버퍼의 거대 다각형 병합 연산 수행
  [결과] FastAPI AI 엔진의 싱글스레드 CPU 100% 프리징 및 광역 연산 지연 초래
- 도로의 방위각·경사도·도로폭을 사전 연산하지 않고, 가상 보행자 2,000명의 교차로 이동마다 삼각·지수함수로 중복 계산함
  [결과] 불필요한 CPU 연산 루프가 누적되어 시뮬레이션 지연 및 응답 속도 저하 초래

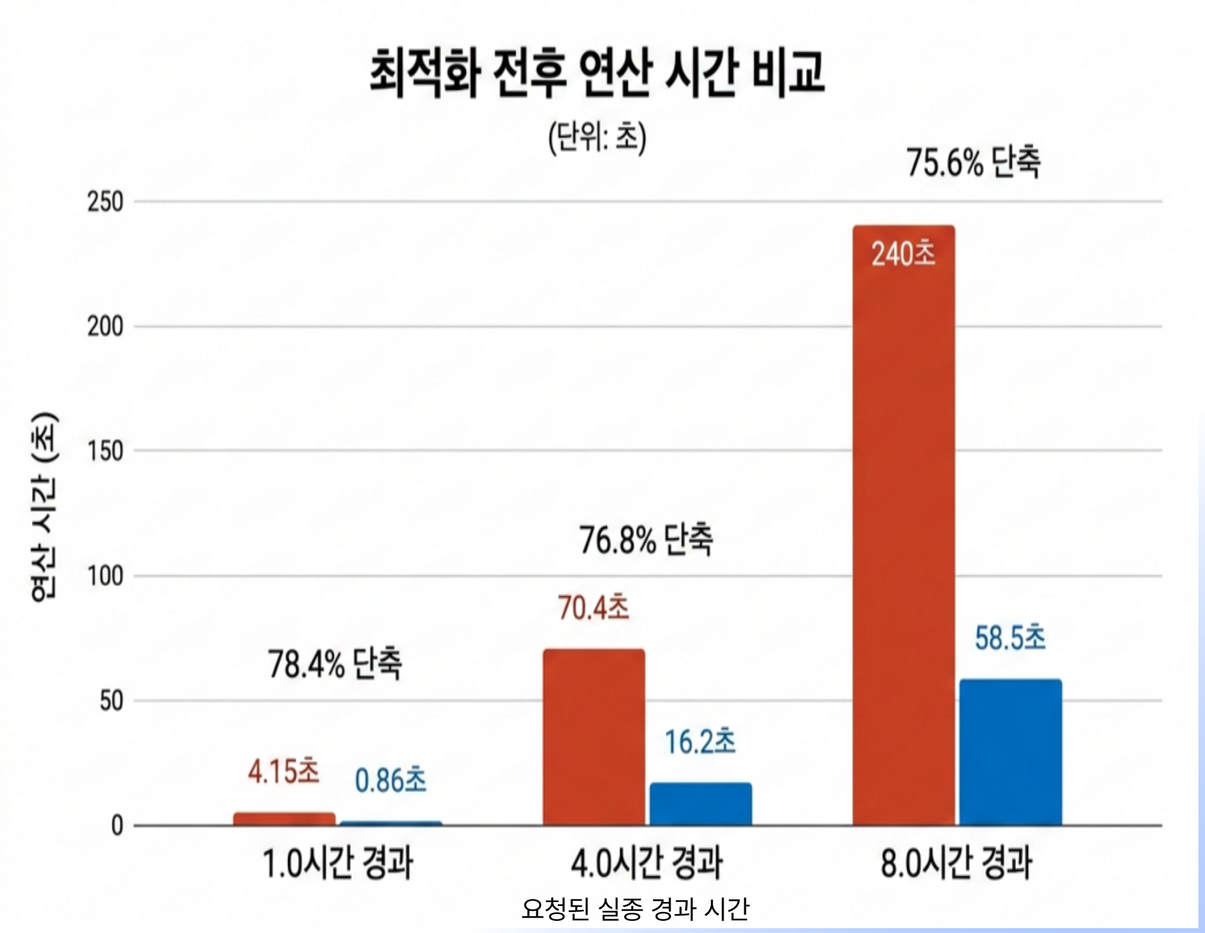
6.3. 대규모 공간 연산 병목 제거

해결 과정

- 불필요한 기하 연산 생략 (화면 표시 최적화)
- 반복 계산 제거 (사전 계산 후 저장)

* 측정 기준: 서울특별시 마포구 도화동 일대

경과 시간	수색범위	최적화 전	최적화 후	지연 감소
1.0시간	약 1.56 km	3.2~5.1초	0.62~1.1초	78.4%
4.0시간	약 6.24 km	70.4초	16.2초	76.8%
8.0시간	약 12.50 km	240초	58.5초	75.6%



07

기술 연구

7.1. 공간 수색 압축 파이프라인

7.2. 지형

7.3 행동 피쳐

7.4. 검증

7.5. 결과

7.1. 공간 수색 압축 파이프라인

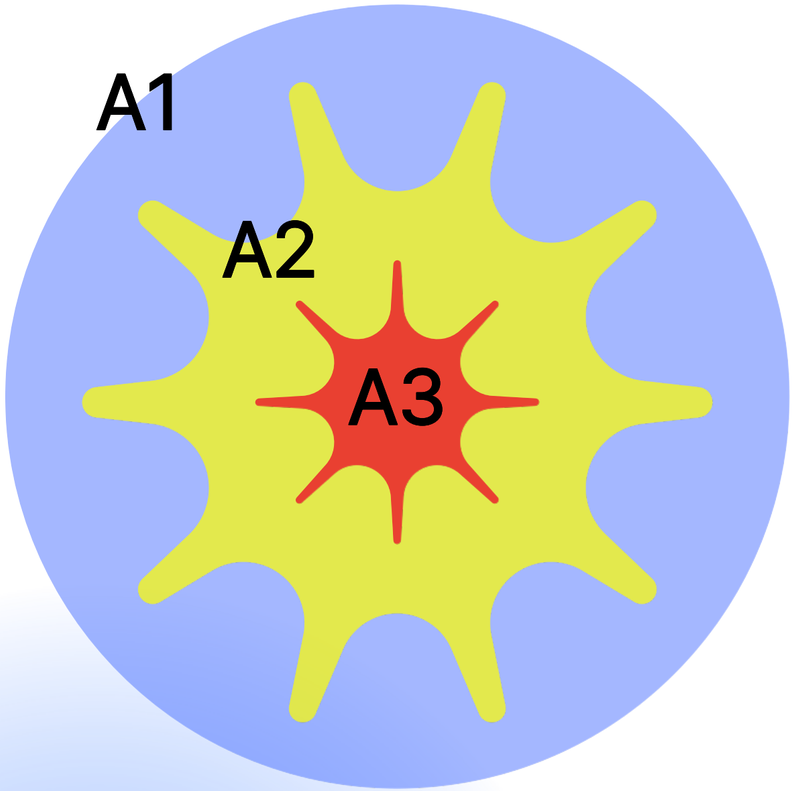
데이터 한계 인식 및 모델링 철학

- 개인정보보호법상 실종자 실시간 GPS 원천 데이터가 부재하여 신경망 역전파 지도학습은 성립할 수 없음
  - 전 세계 수색구조대와 미국 NASAR가 공인 표준으로 채택하고 있는 Robert Koester의 수색 교본《Lost Person Behavior》를 기준으로 설계함

- 분초를 다투는 수색에서 생성형 AI의 환각을 배제하기 위해, 설명 가능한 공간 확률 모델을 채택함

7.1. 공간 수색 압축 파이프라인

3단계 수색 면적 압축 파이프라인(A1→A2→A3)



- 물리적 최대 권역(A1)
  - 기준 보행속도(v0=1.56 km/h)와 경과시간의 단순 등속 원형 권역
- 보행 가능 권역(A2)
  - A1 내의 보행 도로망 중심선에 10m의 폭을 결합해 보행 지형을 추출
- 실제 통과 권역(A3)
  - 2,000명의 가상 에이전트가 행동 모델에 따라 실제로 1회 이상 통과한 도보에 10m의 폭을 조합하여 지형을 추출

7.2. 지형 및 공간 인프라

보행 공간 추출 및 토폴로지 격리

- 원천 데이터
  - 전 세계 표준 오픈 지도인 OpenStreetMap(OSM)에서 보행 네트워크 추출

보행 도로망에 주입한 3대 환경 물리 인자

- 도로 방위각 (Bearing)
  - 터널 시야 및 직진 관성(사잇각 코사인 유사도)을 산출하는 기준축
- DEM 지형 경사도 (Slope)
  - Koester 오르막 기피 효용 감점 및 Tobler 보행 속도 감속의 물리 저항
- 도로 위계 (Hierarchy)
  - 시인성이 확보되는 주간선도로 추종을 유도하는 도로 폭 가중치

7.3. 행동 피쳐 (Behavioral Features)

수색 행동학 교본 기반 임상 행동의 수식화

- 터널 시야(Tunnel Vision) 및 직진 관성
  - 교본에 명시된 치매 환자의 주변 시야 결손, 막힐 때까지 직진하는 특성, 방향 전환 결여를 모델링함.

- 큰길 추종
  - 인지 저하 상태에서 시인성이 확보되는 주간선도로를 따라 걷는 경향을 반영함.

7.3. 행동 피쳐 (Behavioral Features)

갈림길 다항 로짓 효용 모델

- 갈림길 선택 점수(U)=(직진 가산점) + (큰길 가산점) - (오르막 저항) - (이동거리 저항)
  - 직진 가산점 (+): 원래 걷던 방향과 비슷할수록 더해지는 점수
  - 큰길 가산점 (+): 도로 폭이 넓고 시야가 트인 길일수록 더해지는 점수
  - 오르막 저항 (-): 경사가 심할수록 깎여나가는 저항 점수
  - 이동거리 저항 (-): 도로 길이가 길수록 깎여나가는 거리 패널티

- P(ij) = exp(U(ij)) / SUM[exp(U(im))]
  - P(ij) (선택 확률): 에이전트가 현재 교차로 노드 i에서 다음 후보 노드 j로 이어지는 도로를 선택할 확률 (0.0~1.0).
  - U(ij) (도로 효용 점수): 후보 도로 j가 갖는 매력도 점수.
  - exp(U(ij)): 효용 점수를 지수함수로 변환하여 점수가 조금만 높아도 선택 확률이 크게 증가하도록 증폭하고, 모든 값을 양수로 변환합니다.
  - SUM[exp(U(im))] (분모의 합계): 현재 교차로 i에 연결된 모든 진출 가능한 후보 도로의 지수값 총합입니다. 모든 후보 도로의 확률 합이 정확히 1.0이 되도록 나누어 주는 정규화 (Normalization) 역할을 수행합니다.

- 이동 종료 기준: 개인별 이동 지속 시간 소진 또는 막다른 길 도달 시 정지

7.3. 행동 피쳐 (Behavioral Features)

우선 확인 거점(POI) 선별

- 거점 점수화 및 시설물 편향 보정
  - 에이전트의 통과 빈도 및 최종 정지 도로에 맞닿은 실제 서울시 건축물·거점을 객관적으로 추출

- 150m 비최대 억제 (NMS, Non-Maximum Suppression) 공간 분산
  - 목적: 동일 블록 내 인접 거점들의 상위 순위 독점을 방지하고 수색 자원을 효율적으로 분산
  - 방식: 300m 전술 반경 중복을 억제하기 위해 최소 150m 물리적 이격을 강제, 서로 다른 3대 축선 상의 TOP 3 독립 거점을 도출

7.4. 검증

거시 학술 검증의 목적

- 시뮬레이션 결과가 실제 치매 환자의 이동 한계(Mobility Budget)를 통계학적으로 현실성 있게 모사하는지 객관적 기준선(Baseline Prior)으로 대조 검증

검증 대조 지표(거리 분포도)

- 25% 분위수 거리 (Q1): 0.30 km
- 50% 분위수 거리 (Q2, 중앙값): 1.10 km
- 75% 분위수 거리 (Q3): 3.20 km
- 95% 분위수 거리 (Q4): 12.60 km

통계학적 합격 판정 기준

- 문헌 실측 사분위수(0.3 / 1.1 / 3.2 km) 대비 시뮬레이션 표본 오차 ≤ ±20.0% 이내
- 귀무가설(H0): "시뮬레이션 이동 거리 분포와 실제 치매 환자 이동 거리 분포는 동일하다"
  - 판정 임계선: 유의수준 α = 0.05 기준, p-value > 0.05 충족 (동일 모집단 분포 채택)

7.4. 실 사건을 바탕으로 만든 데이터셋과 검증

언론 기사 기반 벤치마크 구축

- 기사 특성상 지번이 가려지고 랜드마크(정류장, 공원 벤치, 상가 등)로 보도됨
- 그렇기 때문에 실제 수색 환경을 반영한 반경 완충 구역(Buffer) 검증 적용

블라인드 검증 프로토콜 및 3대 평가 기준

- 블라인드 테스트: 발견 위치는 완전히 가려둔 채, [출발지 좌표]와 [경과 시간]만으로 시뮬레이션 구동
- 핵심 수색 권역(A3) 포섭
  - 상위 20% 우선 도로망 내 실제 발견 지점 포함 여부 (Hit-Rate)
- 현장 전술 버퍼 매칭
  - 우선 추천 거점 반경 100m(도보 수색) 및 300m 내 포섭 여부
- 이격 거리(D_error)
  - 추천 최우선 거점 중심점과 데이터셋 발견 장소 간 최단 오차 거리 측정

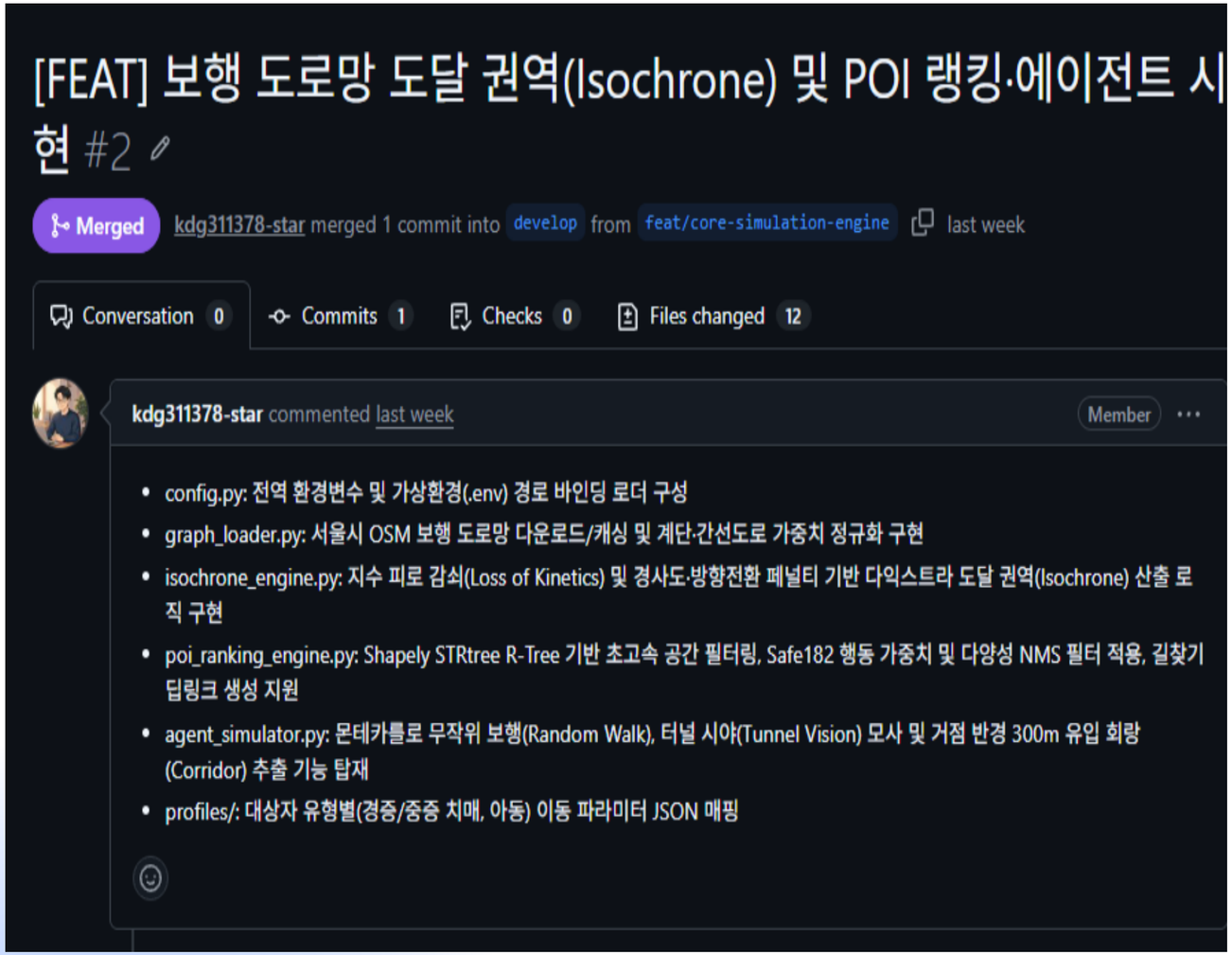
7.5. 기술 연구 - 결과 (Empirical Results & Technical Identity)

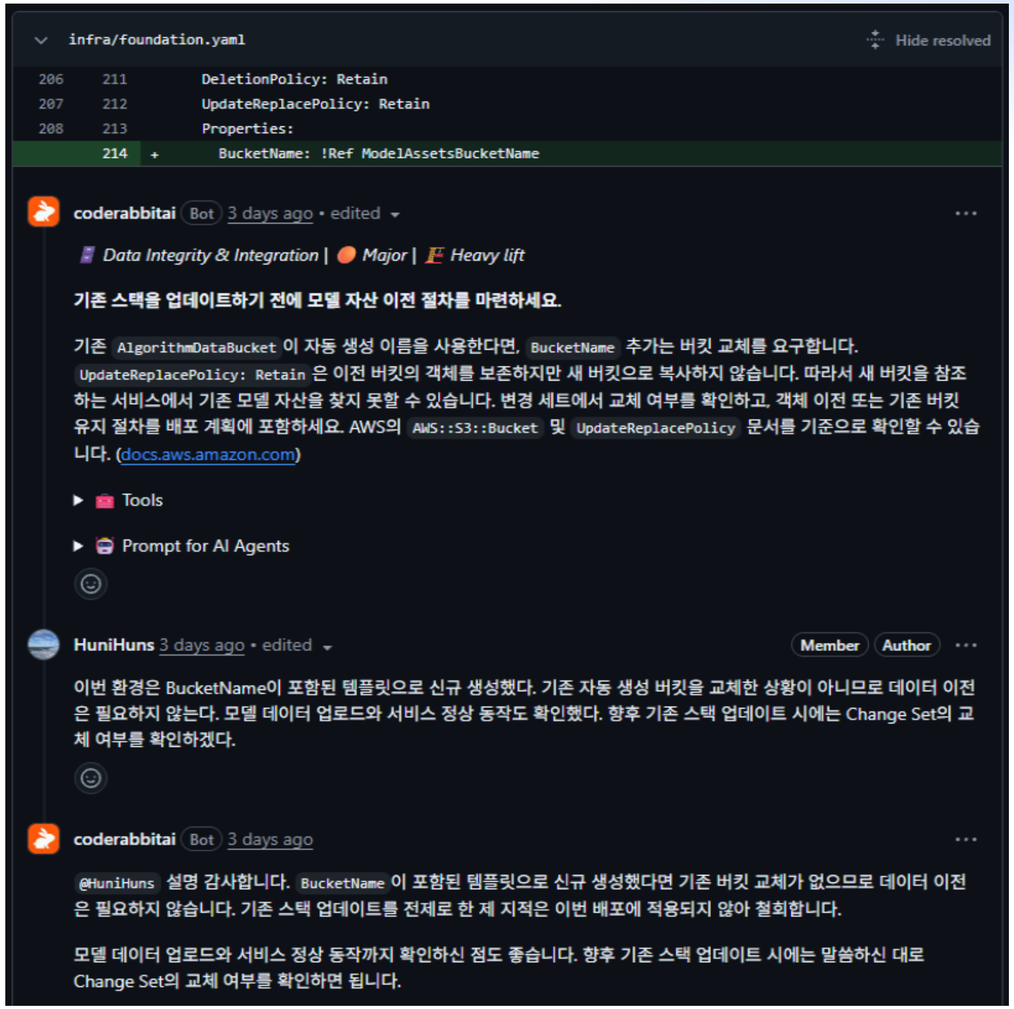
지표	V2(191)	V3 Fact(44)	V3 Aug(176)	V3 전체(220)	목표	최종해석
A3	21.47%	25.00%	16.48%	18.18%	≥70%	전구간목표미달
A3 + 100m	75.92%	72.73%	66.48%	67.73%	≥75%	V2만 PASS V3 계열미달
ARR Total	98.06%	94.78%	94.81%	94.81%	≥70%	명확한 PASS
Brier 300m	0.18118	0.20728	0.19557	0.19791	≤0.200	V3 Fact FAIL
평균 P300	13.09%	19.56%	21.79%	21.35%	-	확률값은우선순위지표 해석이적합
평균 MoE	±1.38%	±1.59%	±1.60%	±1.60%	≤±3%	Monte Carlo 추정오차
D_error 중앙값	838.1m	451.9m	596.6m	580.1m	-	점예측은약함
300m 포섭률	24.08%	29.55%	26.14%	26.82%	-	유효한신호존재
100m 적중률	3.66%	4.55%	5.11%	5.00%	-	낮은수준

08

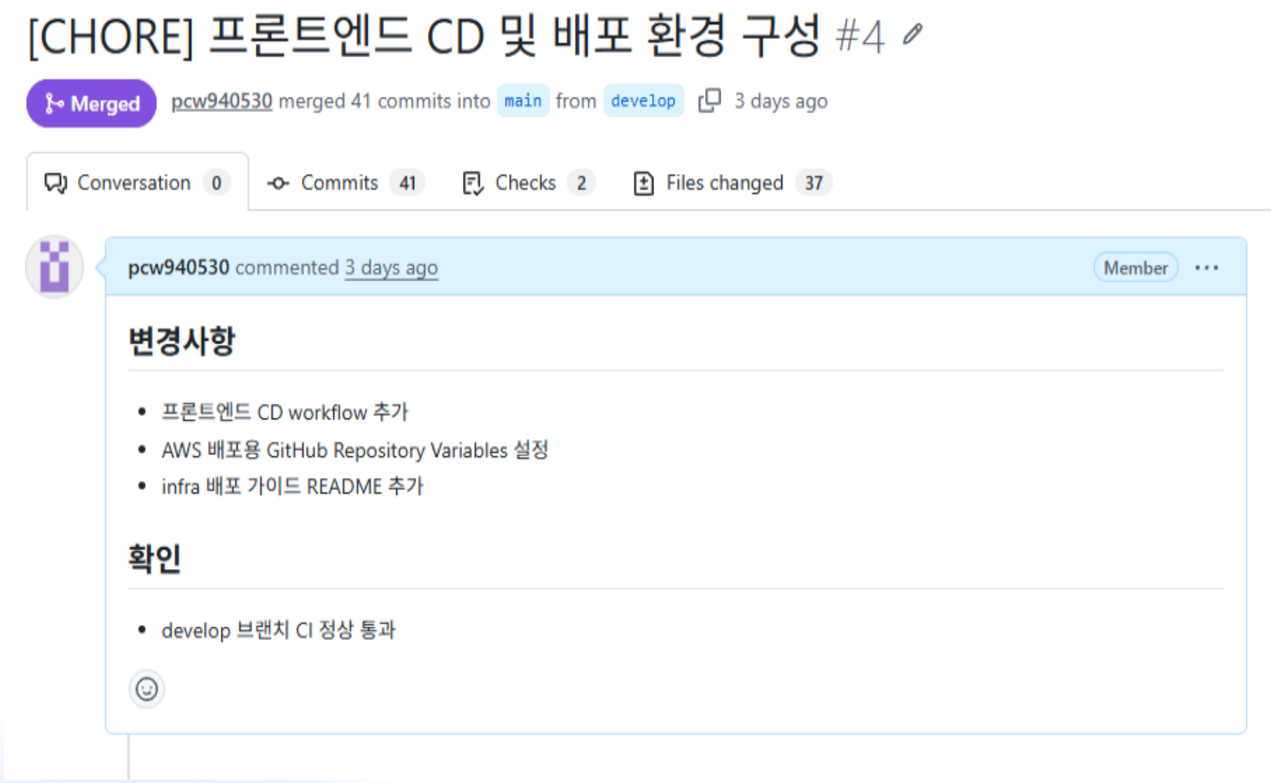
형상관리 및 배포

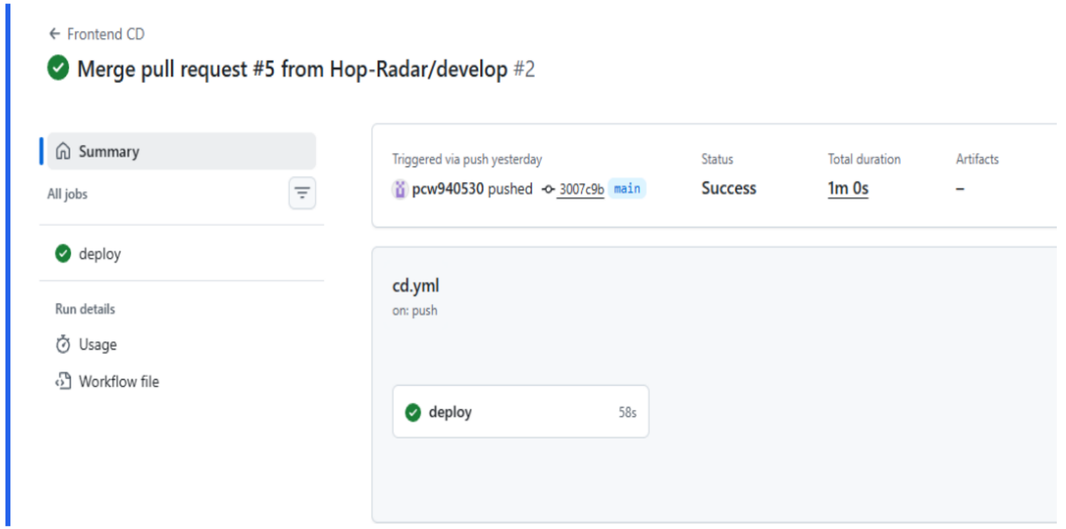
8. 형상관리 및 배포 Github Pull Request를 통한 형상관리





8. 형상관리 및 배포 GitHub Pull Request 및 Actions를 활용한 형상관리·자동 배포





8. 형상관리 및 배포 Docker, Docker-compose를 이용한 AWS 배포

1. 서비스별 Dockerfile 작성

프론트엔드·백엔드·알고리즘 서버의 Dockerfile을 작성하여 각 서비스의 실행 환경을 이미지로 구성

프론트엔드의 Dockerfile

```
FROM nginx:1.27-alpine

COPY . /usr/share/nginx/html
COPY infra/nginx.conf /etc/nginx/conf.d/default.conf
COPY infra/docker-entrypoint.sh /docker-entrypoint.sh

RUN sed -i 's/\r$//' /docker-entrypoint.sh \
    && chmod +x /docker-entrypoint.sh

EXPOSE 80

ENTRYPOINT ["/docker-entrypoint.sh"]
```

백엔드의 Dockerfile

```
FROM eclipse-temurin:21-jre-jammy

WORKDIR /app

RUN groupadd --gid 10001 goldenstep \
    && useradd --uid 10001 \
        --gid goldenstep \
        --no-create-home \
        --shell /usr/sbin/nologin \
        goldenstep

COPY --chown=goldenstep:goldenstep build/libs/goldenstep.jar /app/app.jar

USER goldenstep

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

알고리즘 서버의 Dockerfile

```
# ================================================================================
# [Stage 1] Build Stage: C 확장 컴파일 및 의존성 패키징
# ================================================================================
FROM python:3.11-slim AS builder

WORKDIR /app

# 빌드 스테이지에서 C 확장 컴파일을 위한 gcc 및 libgeos-dev 설치
RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc \
    libgeos-dev \
    && rm -rf /var/lib/apt/lists/*

# requirements.txt 복사 및 패키지 설치 (--user 옵션으로 /root/.local에 설치)
COPY requirements.txt .
RUN pip install --user --no-cache-dir -r requirements.txt

# ================================================================================
# [Stage 2] Runner Stage: 경량 운영 서빙 이미지
# ================================================================================
FROM python:3.11-slim AS runner

WORKDIR /app

# 공간 기하 라이브러리(Shapely C-GEOS 바인딩), 헬스체크용 curl, 그리고 AWS S3 연동용 awscli 설치
RUN apt-get update && apt-get install -y --no-install-recommends \
    libgeos-c1v5 \
    curl \
    awscli \
    dos2unix \
    && rm -rf /var/lib/apt/lists/*

# 빌더 스테이지의 파이썬 패키지 바이너리 복사
COPY --from=builder /root/.local /root/.local
ENV PATH=/root/.local/bin:$PATH
ENV PYTHONUNBUFFERED=1

# 데이터 디렉터리 기본 구조 확보 (마운트 또는 Entrypoint 다운로드 타겟)
RUN mkdir -p data/networks data/pois

# 애플리케이션 핵심 서빙 소스 및 엔트리포인트 복사
COPY core/ ./core/
COPY api/ ./api/
COPY main.py .
COPY infra/docker/entrypoint.sh ./entrypoint.sh

# 엔트리포인트 권한 부여 및 윈도우(CRLF) 줄바꿈 문자 오류 방지
RUN dos2unix ./entrypoint.sh && chmod +x ./entrypoint.sh

EXPOSE 8000

# Docker 네이티브 Health Check (초기 엔진 적재 40초 대기)
HEALTHCHECK --interval=10s --timeout=5s --start-period=40s --retries=3 \
  CMD curl -f http://localhost:8000/health || exit 1

# 컨테이너 시작 시 엔트리포인트 스크립트를 통해 AWS S3 동기화 후 Uvicorn 가동
ENTRYPOINT ["./entrypoint.sh"]
```

8. 형상관리 및 배포 Docker, Docker-compose를 이용한 AWS 배포

2. Docker Compose 설정

Docker Compose로 3개 서비스의 **네트워크·환경변수·볼륨·실행 순서**를 정의. Nginx의 **80·443 포트만 외부에 공개**하고, 백엔드와 알고리즘 서버는 내부 네트워크로 연결하였다.

compose.prod.yaml

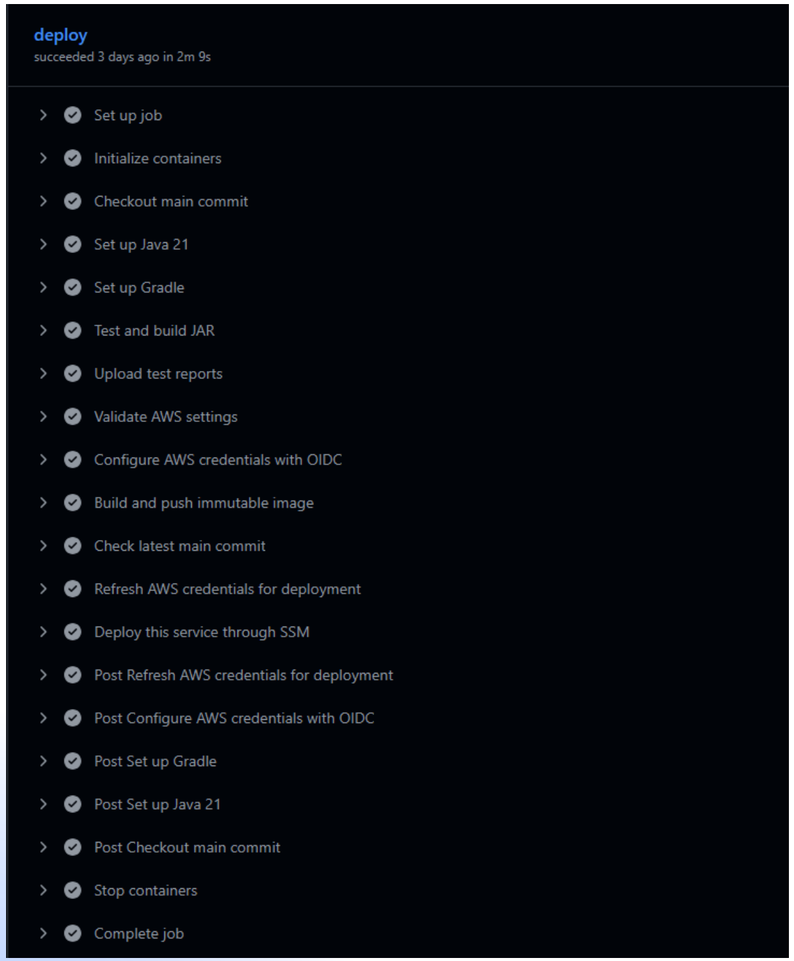
```yaml
services:
  frontend:
    image: ${FRONTEND_IMAGE:?FRONTEND_IMAGE is required}
    container_name: gs-frontend-web
    restart: unless-stopped

    env_file:
      - path: ${FRONTEND_ENV_FILE:?FRONTEND_ENV_FILE is required}
        required: true
        format: raw

    ports:
      - "80:80"
      - "443:443"

    volumes:
      - type: bind
        source: ${NGINX_CONFIG_PATH:?NGINX_CONFIG_PATH is required}
        target: /etc/nginx/conf.d/default.conf
        read_only: true
        bind:
          create_host_path: false

      - type: bind
        source: ${TLS_CERT_DIR:?TLS_CERT_DIR is required}
        target: /etc/letsencrypt
        read_only: true
        bind:
          create_host_path: false

    depends_on:
      backend:
        condition: service_started

    networks:
      - goldenstep
```

```yaml
  backend:
    image: ${BACKEND_IMAGE:?BACKEND_IMAGE is required}
    container_name: gs-backend-api
    restart: unless-stopped

    expose:
      - "8080"

    env_file:
      - path: ${BACKEND_ENV_FILE:?BACKEND_ENV_FILE is required}
        required: true
        format: raw

    environment:
      SPRING_PROFILES_ACTIVE: infra,fastapi
      JAVA_TOOL_OPTIONS: -Xms256m -Xmx1024m
      AI_BASE_URL: http://algorithm:8000
      TZ: Asia/Seoul

    depends_on:
      algorithm:
        condition: service_healthy

    stop_grace_period: 30s

    networks:
      - goldenstep

    logging: *default-logging
```

```yaml
  algorithm:
    image: ${ALGORITHM_IMAGE:?ALGORITHM_IMAGE is required}
    container_name: gs-ai-engine
    restart: unless-stopped
    init: true

    expose:
      - "8000"

    environment:
      TZ: Asia/Seoul
      CACHE_GRAPH_PATH: /app/data/networks/seoul_walk_network.graphml
      BUILDING_POI_DATA_PATH: /app/data/pois/seoul_building_pois_3.parquet
      PUBLIC_POI_DATA_PATH: /app/data/pois/seoul_public_pois.geojson

    volumes:
      - type: bind
        source: ${AI_DATA_DIR:?AI_DATA_DIR is required}
        target: /app/data
        read_only: true
        bind:
          create_host_path: false

    command:
      - uvicorn
      - main:app
      - --host
      - 0.0.0.0
      - --port
      - "8000"
      - --workers
      - "1"
```

8. 형상관리 및 배포 Docker, Docker-compose를 이용한 AWS 배포

3. GitHub Actions를 통한 이미지 빌드



4. Amazon ECR에 이미지 업로드

8. 형상관리 및 배포 Docker, Docker-compose를 이용한 AWS 배포

5. EC2의 서비스 컨테이너 교체

```
Digest: sha256:2d9c215dbb11fd172af3f84889e833fe20b1c481ed5968b4071795a1c58f920f
Status: Downloaded newer image for 593011452419.dkr.ecr.ap-northeast-2.amazonaws.com/goldenstep-backend:39a0facb196bd7441ec816e48d4324b11b17cff3
593011452419.dkr.ecr.ap-northeast-2.amazonaws.com/goldenstep-backend:39a0facb196bd7441ec816e48d4324b11b17cff3
NAME             IMAGE                                                                                                COMMAND                  SERVICE   CREATED          STATUS          PORTS
gs-backend-api   593011452419.dkr.ecr.ap-northeast-2.amazonaws.com/goldenstep-backend:39a0facb196bd7441ec816e48d4324b11b17cff3   "java -jar /app/app.…"   backend   12 seconds ago   Up 11 seconds   8080/tcp
backend deployment: SUCCESS
```

```
 Container gs-backend-api  Recreate
 Container gs-backend-api  Recreated
 Container gs-backend-api  Starting
 Container gs-backend-api  Started
```

6. 배포 상태 확인 및 이전 이미지 복원

```
[root@ip-10-20-1-205 goldenstep]# docker compose --env-file .env.prod -f compose.prod.yaml ps
NAME              IMAGE                                                                                                COMMAND                  SERVICE     CREATED         STATUS
gs-ai-engine      593011452419.dkr.ecr.ap-northeast-2.amazonaws.com/goldenstep-ai:c66cf77fbfeb50ee1c429b74a93788d4ec655408          "./entrypoint.sh uvi…"   algorithm   10 hours ago    Up 10 hours (healthy)
gs-backend-api    593011452419.dkr.ecr.ap-northeast-2.amazonaws.com/goldenstep-backend:e21904b8250b72dc71d0481169aec71cbdecd3f4    "java -jar /app/app.…"   backend     39 hours ago    Up 39 hours
gs-frontend-web   593011452419.dkr.ecr.ap-northeast-2.amazonaws.com/goldenstep-frontend:3007c9bdd57639b475a7b5c284c6ba75f15897e1   "/docker-entrypoint.…"   frontend    38 hours ago    Up 38 hours
]:80->80/tcp, 0.0.0.0:443->443/tcp, [::]:443->443/tcp
```

09

회의록

9. 회의록 회의록 원문

[26.09.23] 3차 project kick-off 및 planning 회의

날짜: 2026.09.23 10:00 ~ 12:50 | 참여자: 이재훈, 김동규, 박철우 | 장소: 더조은 903호

팀명 : 루트파인더(가칭)
프로젝트명:골든스텝(가칭)

<프로젝트 주제>
보행 네트워크 및 공간 통계 기반 초기 실종자 탐색 의사결정 지원 플랫폼
<기능 요약>
위치 및 시각 입력 인터페이스
보호자가 지도 클릭이나 도로명·지번 주소 검색을 통해 실종자의 마지막 확인 위치를 직관적으로 특정하고, 경과 시각과 대상자 특성(일반 성인, 아동, 치매 노인 등)을 간편하게 설정할 수 있는 입력 폼을 제공합니다
안전 안내 및 법적 고지 모달
서비스 진입 시 "112 경찰 신고가 최우선"임을 필수적으로 고지하고, 본 시스템이 인공지능 환각을 배제한 데이터 기반의 추정 보조 도구임을 명시하여 사용자의 동의를 거친 후 분석 기능을 활성화합니다
보행 도로망 기반 실제 도달 구역**(Isochrone)** 표출
기존 상용 지도의 단순 원형 직선거리 동심원을 탈피하여, 오픈스트리트맵(OSM)의 실제 보행 가능 도로망을 분석해 경과 시간 내 도보로 다다를 수 있는 물리적 도달 가능 영역(Isochrone Polygon)을 지도상에 음영 처리하여 렌더링합니다
결정론적 점수 기반 우선 확인 거점**(POI)** 순위 추천
접근성과 대중교통 정류장, 24시 편의점, 도시공원 등 체류 가능 시설의 특성을 종합한 수학적 점수식을 바탕으로, 실종자가 머무르거나 지나쳤을 가능성이 높은 유력 장소를 상위 1~3순위로 정렬하여 마커와 리스트로 안내합니다
시간대별 영역 확장 슬라이더
30분, 1시간, 3시간 등 경과 시간에 따른 도달 구역의 단계적 확장과 후보 거점의 동적 변화를 직관적인 슬라이더 및 탭 컨트롤을 통해 반응형으로 전환하여 확인할 수 있도록 지원합니다
시간 경과 신뢰도 경고 배지
실종 경과 시간이 장기화(3시간 이상)될 경우 대중교통 탑승이나 원거리 이탈 등 통제 불가능한 변수가 급증함을 반영하여, 화면 상단에 "신뢰도 낮음 / 참고용" 경고 배지를 자동으로 표출해 보호자의 과도한 맹신과 수색 왜곡을 방지합니다
외부 상용 지도 보행 길찾기 딥링크
추천 거점을 선택했을 때 카카오맵 또는 네이버지도의 '도보 길찾기' 화면으로 원클릭 연결(모바일 앱 스킴 및 웹 URL)되도록 하여, 자체 내비게이션 개발 리스크를 줄이면서 현장 탐색 인원의 즉각적인 이동을 보장합니다
긴급 **112** 원터치 신고 연동
모바일 뷰포트 우측 하단에 상시 고정되는 플로팅 액션 버튼(FAB)을 배치하여, 위급 상황 발생 시 지체 없이 단말기의 전화 다이얼러(`tel:112`)를 호출해 공권력에 신속히 신고할 수 있도록 돕습니다
탐색 현황 요약 카드 모달 브리핑
기준 위치 좌표, 경과 시간, 예상 도달 면적 규모, 상위 1~3순위 유력 거점 정보를 한눈에 파악할 수 있는 단일 카드 UI로 정리해 제공함으로써, 패닉 상태의 보호자가 상황을 즉시 파악하고 타인에게 구두나 텍스트로 쉽게 전달할 수 있게 합니다

[26.09.30] 3차 Daily Scrum — Sprint 1

날짜: 2026.09.30 09:32 ~ 10:00 | 참여자: 이재훈, 김동규, 박철우 | 장소: 더조은 903호

진행 상황
- Frontend : 경로 탐색(도보 API), 탐색 범위 원형 표시, 탐색 결과와 공유 페이지 통일, 입력 값 부재 예외처리
- AI(알고리즘) : 치매 노인 개발 완료, 수치 유의미한지 정리

피드백
- 신체적 특징, 장소적 특징 : Back -> ML의 흐름이 아닌 프론트에서 탐색 결과와 공유 화면에 추가 정보를 제공
- 탐색 종료 : 동적 버튼 활성화, 최초 1회 자동 모달창 제공, 이후는 버튼으로 모달창 제공, 선택지는 탐색 종료 후 새 탐색 시작 or 해당 탐색 머무르기

목표
- Frontend : 피드백 내용 반영, 백엔드 지역 검색 응답 예시 반영
- Backend : 분석 결과 api, 네이버 지도 api 연결해서 좌표값으로 주소 불러오기
- AI(알고리즘) : 발표에 쓰일 수 있도록 수치 조정, 최종 response와 목업 비교 후 정리

9. 회의록

회의록 원본

[26.10.04] Sprint 1 Review

날짜: 2026.10.04 22:05 ~ 23:10 | 참여자: 이재훈, 김동규, 박철우 | 장소: 디스코드(비대면)

Review

1. 모바일 반응형 웹페이지가 제대로 동작하지 않음
2. 공유 스냅샷 API 개발 필요
3. 서비스에 대한 안내 부족 → 사용자가 익숙치 않은 서비스 때문이라 예상

Sprint 1 Backlog

1. 모바일 웹 사용자가 112 모달 버튼을 확인할 수 있도록 한다.
2. 공유 스냅샷을 공유하면 Mock 데이터가 아닌 실제 분석 데이터가 나오도록 한다.
3. 사용자가 장소명을 검색하면 입력한 내용 기반 후보지를 5개 추천한다는 사실을 인지하도록 한다.

결과 및 최종 보고서 역할 분담
사용자 피드백 : 10/4일 23시 설문 종료.

1. 결과 보고서(화요일 수업 전까지)
   - 프로젝트 개요 : B1 | E1, E2 결과 활용
   - 추후 개선 사항 : B3
   - 사용자 피드백 : F3, F4
2. 최종 보고서
   - 숫자 데이터 평균 값 정리 및 각 질문 별 만족도 확인 (박철우)
     : 각 질문 별 점수 평균 | 각 목차(A, B, C …) 별 점수 평균 (단, 부정적인 질문인 경우 1을 5로, 5를 1로 계산)

[26.10.06] 6차 Daily Scrum — Sprint 2

날짜: 2026.10.06 10:15 ~ 10:26 | 참여자: 이재훈, 김동규, 박철우 | 장소: 더조은 903호

Sprint 1 백로그 적용 확인(피드백 적용에 작성 예정) : 개발 종료

해야할 일

- 시연 영상 녹화 : 웹(김동규), 모바일(박철우)
- 사용자 피드백 통계 그래프 모양 및 index 수정
- 추가 결과보고서 작업 진행, 결과보고서 완성 후 발표 순서 및 발표 자료 작성

10

자체 평가 의견

10.1. [프로젝트] 사용자 피드백 기반 자체 평가

10.2. [프로젝트] 잘한 점과 아쉬운 점

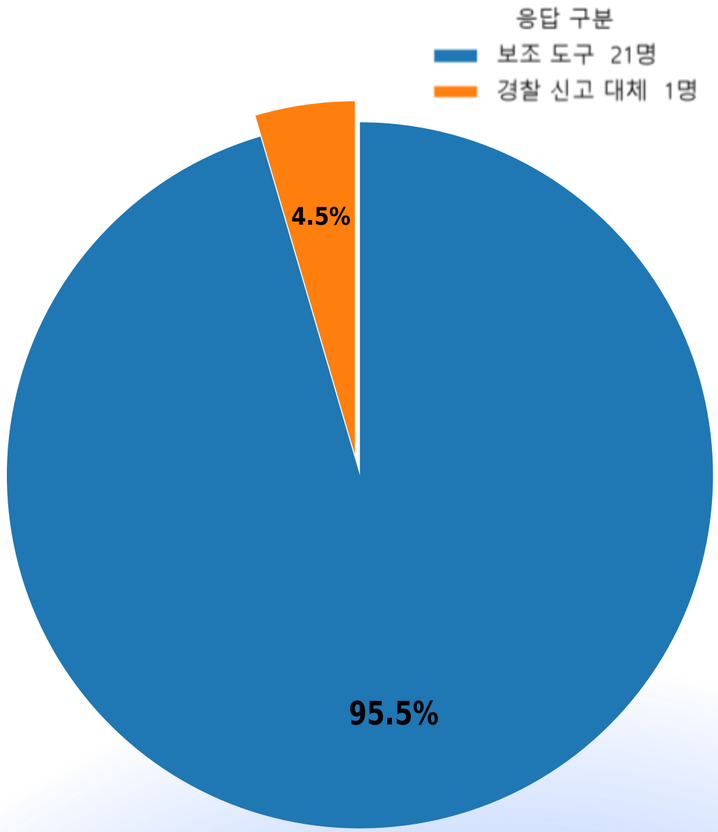
10.3. [프로젝트] 추후 개선 사항

10.4. 소감

10.1. [프로젝트] 사용자 피드백 기반 자체 평가 기획 의도

Q. 골든스텝의 역할에 대한 설명으로 가장 적절한 것은 무엇이라고 생각하십니까?

답변	응답 수	비율(%)
신고 이후 주변을 확인할 때 참고할 수 있는 보조 도구이다.	21	95.5
경찰 신고를 대신하는 서비스이다.	1	4.5



10.2. [프로젝트] 잘한 점과 아쉬운 점

잘한 점

- 서울시 전체 도로 링크의 불변 기하 속성을 서버 기동 시 **인메모리에 정적으로 주입**하고, 실시간 API 응답에서는 불필요한 기하 합집합 생성을 건너뛰도록 **아키텍처를 이원화**했다.
- 초기에 하드코딩했던 정성적 설명을 미시 **행동규칙에서 분리**하고, 벡터·경사·도로위계 등 지도에서 **반복 산출 가능한 물리량으로 함수화하는 리팩토링**을 진행했다.
- AI 에이전트를 활용한 코드 리뷰로 **구현 코드의 잠재적 오류와 개선 사항을 점검**하며 **2차 검증**을 수행했다.

아쉬운 점

- 개인정보 보호 여건으로 실종자의 초 단위 연속 GPS 이동 궤적 공개 데이터가 없기 때문에, 실제 기사 및 경찰 발표의 거리·랜드마크를 역지오코딩하고 공간 완충구역을 사용하여 사후 간접 검증하는 수준에 머물렀다.
- 기획 단계에서 시간이 많이 소모되어 기획했던 기능 중 구현하지 못한 기능이 존재했다.
- 개발 초기부터 API 응답 구조와 상태 변화를 충분히 고려했다면 불필요한 수정 작업을 줄일 수 있었을 것이다.

10.3. [프로젝트] 추후 개선 사항

구분	AS-IS	TO-BE
공간 커버리지의 전국 확대	서울 외 타 시·도 지역 질의 시 공간 분석 지원이 제한	온디맨드(On-demand) 서브그래프 동적 버퍼링 파이프라인을 구축
다중 대상군(분류군)으로의 도메인 확장	치매 노인 단일 도메인 위주로 엔진을 구축·검증	다중 분류군별 거동 모델을 플러그인 형태로 단계적 확장
지형 분석 고도화 (등산로 및 건물 내부 수직 공간 반영)	지상 2D 평면 보행 도로망(OSM) 선형을 중심으로 모델링	3D 입체 지형·실내 복합 공간 분석 엔진으로 고도화
공유 스냅샷 기능 고도화	분석 진행한 사용자의 분석 결과를 스냅샷해 제공	공유 받은 사용자도 도보 경로 등 세부 결과 내용 제공

10.4. 소감 - 이재훈

이번 프로젝트에서 팀장을 맡으며 담당 기능뿐 아니라 팀 전체의 방향과 진행 상황을 살피고, 끝까지 완성해야 한다는 책임감을 느꼈습니다. 제 판단이 개발 일정과 팀원들의 작업에 영향을 준다는 점에서 **의견을 듣고 우선순위를 조율하는 과정의 중요성을 체감**했습니다.

기획 당시에는 더 좋은 서비스를 만들고자 기능의 필요성, 사용자 경험, 구현 가능성과 예외 상황까지 많은 요소를 고려했습니다. 충분히 검토한 뒤 결정하려다 기획이 계속 미뤄졌고, 개발 시간도 줄어들었습니다. 이를 통해 **핵심 목적과 필수 기능을 먼저 정하고, 나머지는 우선순위에 따라 구체화**해야 한다는 것을 배웠습니다.

개발 중에도 **선택한 방식이 맞는 방향인지 스스로 질문**했습니다. 특히 정보 입력 세션을 DB에 저장하는 것이 적절한지, 어떤 정보를 얼마 동안 보관할지 고민했습니다. 세션 복원과 데이터 관리의 필요성을 고려해 24시간 보관 후 삭제하도록 구현하고, 입력 검증과 예외 처리, 서버 연동과 배포 설정을 점검하며 **더 효율적이고 안정적인 서비스를 만들고자 노력**했습니다.

짧아진 일정에 대응하기 위해 **팀원들과 더 자주 소통**하고자 노력했습니다. 데일리 스크럼으로 진행 상황과 어려움을 공유하고, 서로의 작업이 연결되는 지점을 확인하며 조율했습니다. 여러 시행착오에도 함께 문제를 해결해 끝까지 프로젝트를 완성하면서 협업의 힘과 맡은 역할을 다하는 태도의 중요성을 배웠습니다.

기획서와 WBS, 구현 방식과 배포 과정을 정리하면서 **문서가 팀의 공통 이해를 만들고 후속 작업을 이어가는 기준**임을 체감했습니다. 앞으로도 실무에서 설계의 근거와 작업 과정을 기록하며, 소통과 문서화를 개발의 일부로 실천하는 개발자가 되겠습니다.

10.4. 소감 - 김동규

이번 프로젝트는 정제된 정형 데이터를 수동적으로 분석하던 기존 방식에서 벗어나, 문헌 속 거시 행동 통계 이론을 실제 보행 시뮬레이션 알고리즘으로 직접 모델링하고 데이터로 타당성을 검증해 본 뜻깊은 경험이었습니다. 초기에는 문헌의 정성적 서술을 자의적인 수치로 옮겼다가 시뮬레이션이 과확산되는 왜곡을 겪으며, 연구와 구현에는 **주관적 직관이 아닌 엄밀한 기준선과 객관적 검증 지표가 필수적**임을 체감했습니다.

무엇보다 **데이터가 존재하지 않는 한계를 공학적으로 극복**하며 큰 성장을 이루었습니다. 개인정보보호법상 실종자의 실제 이동 GPS 궤적을 구할 수 없는 제약 속에서, 무의미한 가상 난수를 만드는 대신 실제 사건 보도를 수집·지오코딩하고 공간 증강 기법을 도입해 검증 벤치마크 데이터셋을 직접 구축했습니다. 이를 통해 주어진 데이터를 다루는 데 머무르지 않고, **문제 해결에 필요한 평가 데이터를 주도적으로 기획하고 생성하는 역량**을 길렀습니다.

또한 **시스템 간 인터페이스(API) 정의와 소통의 중요성**을 깊이 배웠습니다. AI 엔진의 연산이 정교하더라도 백엔드, 프론트엔드와 사전에 약속된 데이터 스키마와 입출력 규격이 맞지 않으면 서비스가 정상 동작하지 않는다는 점을 겪었습니다. 이를 해결하며 모델 자체의 완성도만큼이나 **사전 명세 합의와 타 파트에 대한 기술적 이해를 바탕으로 한 협업**이 중요하다는 것을 깨달았습니다.

결국 이번 경험을 통해 이상적인 모델링을 넘어, 데이터 수집부터 시스템 간 인터페이스와 실제 화면까지 유기적으로 맞물려야만 진정한 가치가 완성된다는 점을 배웠습니다. 앞으로도 **복잡한 제약 앞에서 편법이나 자의적 타협에 기대지 않고 정직한 공학적 대안을 모색하며, 성능을 부풀리지 않고 데이터가 입증하는 범위 내에서 문제를 해결하는 엔지니어로 성장하겠습니다.**

10.4. 소감 - 박철우

이번 프로젝트는 제가 처음으로 경험한 팀 프로젝트였습니다. 프로젝트를 시작할 당시에는 개발 과정이나 협업 방식에 대해 모르는 것이 많아 팀원들에게 의존해야 하는 부분도 많았습니다. 하지만 **모른다는 사실에 주눅 들기보다, 모르는 것은 솔직하게 질문하고 제가 무엇을 해야 하는지 적극적으로 확인하는 자세**로 프로젝트에 참여했습니다.

맡은 업무의 크기와 관계없이 팀에 피해를 주지 않고 **제 역할을 끝까지 수행하겠다는 책임감**을 가지고 작업했습니다. 혼자 판단하기 어려운 부분은 그대로 진행하기보다 중간중간 팀장님에게 방향을 확인받았고, 피드백을 반영하며 하나씩 결과물을 완성해 나갔습니다. 이러한 과정을 통해 처음에는 도움을 받으며 시작했지만, 점차 제가 맡은 프론트엔드 영역을 이해하고 **스스로 해결할 수 있는 범위도 넓어졌습니다.**

무엇보다 이번 프로젝트를 통해 협업에서 소통이 얼마나 중요한지 깊이 느꼈습니다. 처음에는 소통을 단순히 질문하고 의견을 잘 전달하는 능력이라고 생각했지만, 실제 개발 과정에서는 상대방의 설명을 이해하고 정확한 질문을 하기 위해서도 기술적인 이해가 필요했습니다. 프론트엔드와 백엔드, AI가 서로 연결되는 과정에서 **좋은 소통은 단순한 커뮤니케이션 능력이 아니라 서로의 기술과 작업에 대한 이해를 바탕으로 이루어진다는 점**을 배웠습니다.

이번 경험을 통해 **모르는 것을 숨기는 것보다 적극적으로 질문하고 배우는 태도**, 맡은 역할을 끝까지 책임지는 자세, 그리고 원활한 협업을 위해 지속적으로 기술적 이해를 넓혀가는 것이 중요하다는 것을 배웠습니다. 앞으로도 새로운 환경에서 모르는 것을 두려워하지 않고 적극적으로 배우며, **팀이 신뢰하고 함께 일할수 있는 구성원으로 성장하고 싶습니다.**

Golden Step

Thank you